



实验预备课

2025年秋

实验预备课

- 可编程逻辑器件介绍
- 硬件编程方法与原则
- 硬件开发流程
- 我们的实验系统
- SystemVerilog 的高级语言特性
- Lab2指导
- 仿真

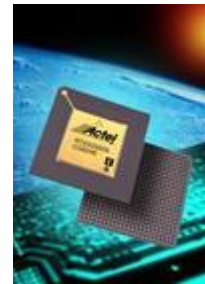
可编程逻辑器件设计

- 可编程器件简介
- 设计原则
- 设计流程

可编程器件简介

□ 概述

- PLD是电子设计领域中最具活力和发展前途的一项技术。
- PLD能做什么呢？可以毫不夸张的讲，PLD能完成任何数字器件的功能，上至高性能CPU,下至简单的位片电路，都可以用PLD来实现。
- 目前有多家公司生产CPLD/FPGA，主要有：ALTERA(Intel)，XILINX，Lattice，Actel 。



可编程器件

□ FPGA

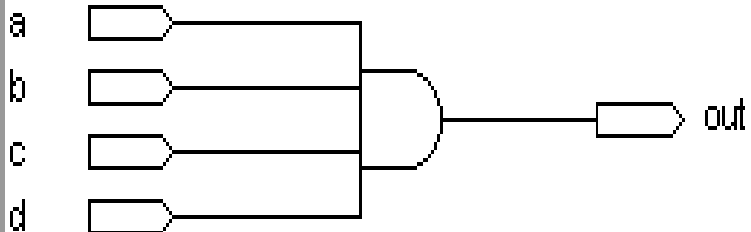
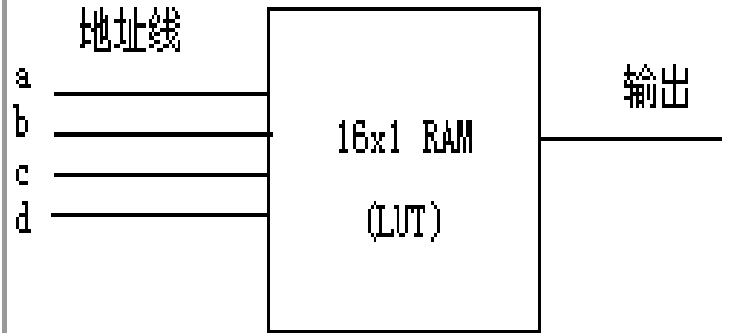
- Field Programmable Gate Array 现场可编程门阵列
- FPGA基于SRAM的架构，集成度高，以LE（包括查找表、触发器及其他）为基本单元，有内嵌Memory、DSP等，支持IO标准丰富。

查找表

□ 基于查找表（Look-Up-Table)的原理与结构:

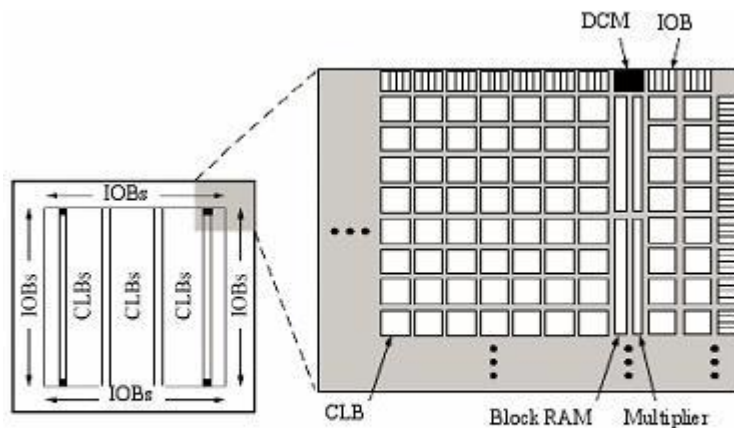
- 采用这种结构的PLD芯片如altera的ACEX,APEX系列,xilinx的Spartan,Virtex系列等。
- 查找表（Look-Up-Table)简称为LUT，LUT本质上就是一个RAM
- 工作原理
 - 当用户通过原理图或HDL语言描述逻辑电路
 - 软件会自动计算逻辑电路的所有可能的结果，并把结果事先写入RAM
 - 每输入一个信号进行逻辑运算就等于输入一个地址进行查表，找出地址对应的内容，然后输出即可。

4输入与门

实际逻辑电路		LUT的实现方式	
			
a, b, c, d 输入	逻辑输出	地址	RAM中存储的内容
0000	0	0000	0
0001	0	0001	0
....	0	...	0
1111	1	1111	1

XilinxFPGA主要部件

- 可编程输入输出单元(IOB)
- 可编程逻辑块(CLB)
- 时钟管理模块(DCM)
- 片内RAM(BRAM)
- 布线资源
- 内嵌功能单元
- 内嵌硬核



Spartan-III 系列结构

Xilinx公司产品概述

□ FPGA

- Virtex 系列
- Spartan器件系列



□ CPLD

- XC9500系列
- CoolRunner系列



□ 其他

- 配置器件SPROM (S系列 P系列)
- IP核

典型的应用领域

□ 数据采集

- 逻辑接口
- 电平接口
 - 电平不同
 - 单端到差分

□ 数字信号处理

□ IC设计验证

□ 其他类，消费，医疗，工业控制

□ 数字信号的基本上都可以用PLD实现



- 摄像机接口
- 视频传输
- 视频处理
- 视频存储
- 视频缩放
- 显示标准转换
- 外设接口

病人监视器

- 显示控制
- 图形生成
- 无线连接



- 交合逻辑
- 主要过程控制 (Main Processing Control)

输液泵

- 胶合逻辑
- 主要过程控制
- 显示控制



手术机器人

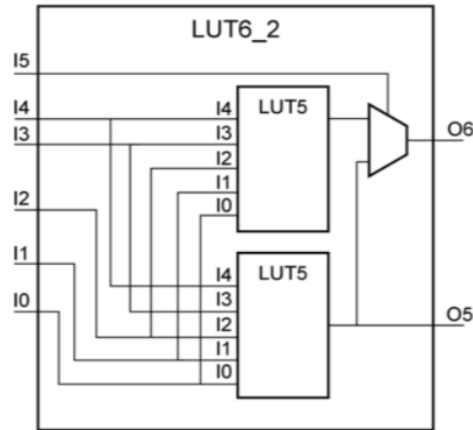
- 摄像机接口
- 视频传输
- 视频处理
- 视频存储
- 3-D视频
- 音频处理
- 电机控制



发展趋势

- 高密度，大容量，高速度
- 低成本，低电压，微功耗，微封装
- 基于IP的设计方法
 - FPGA厂家
 - 开源硬件组织
- 动态可重构
 - 通信系统
 - 重构计算机

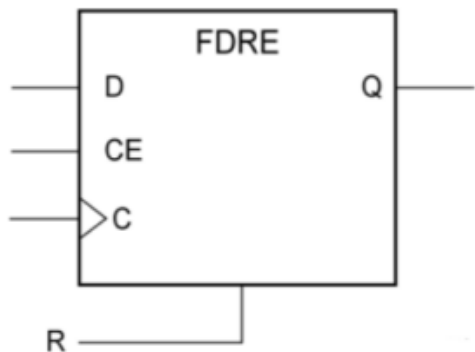
LUT结构



□ 5输入，2输出

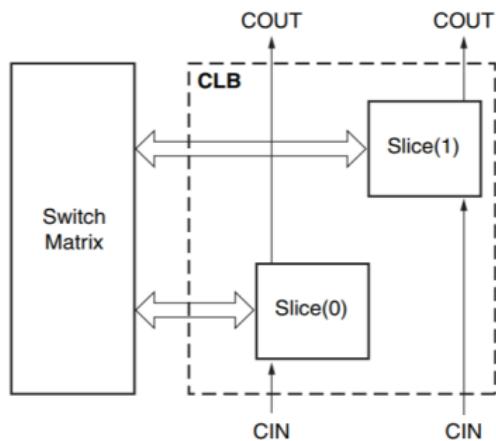
□ 6输入，1输出

FF结构（寄存器）

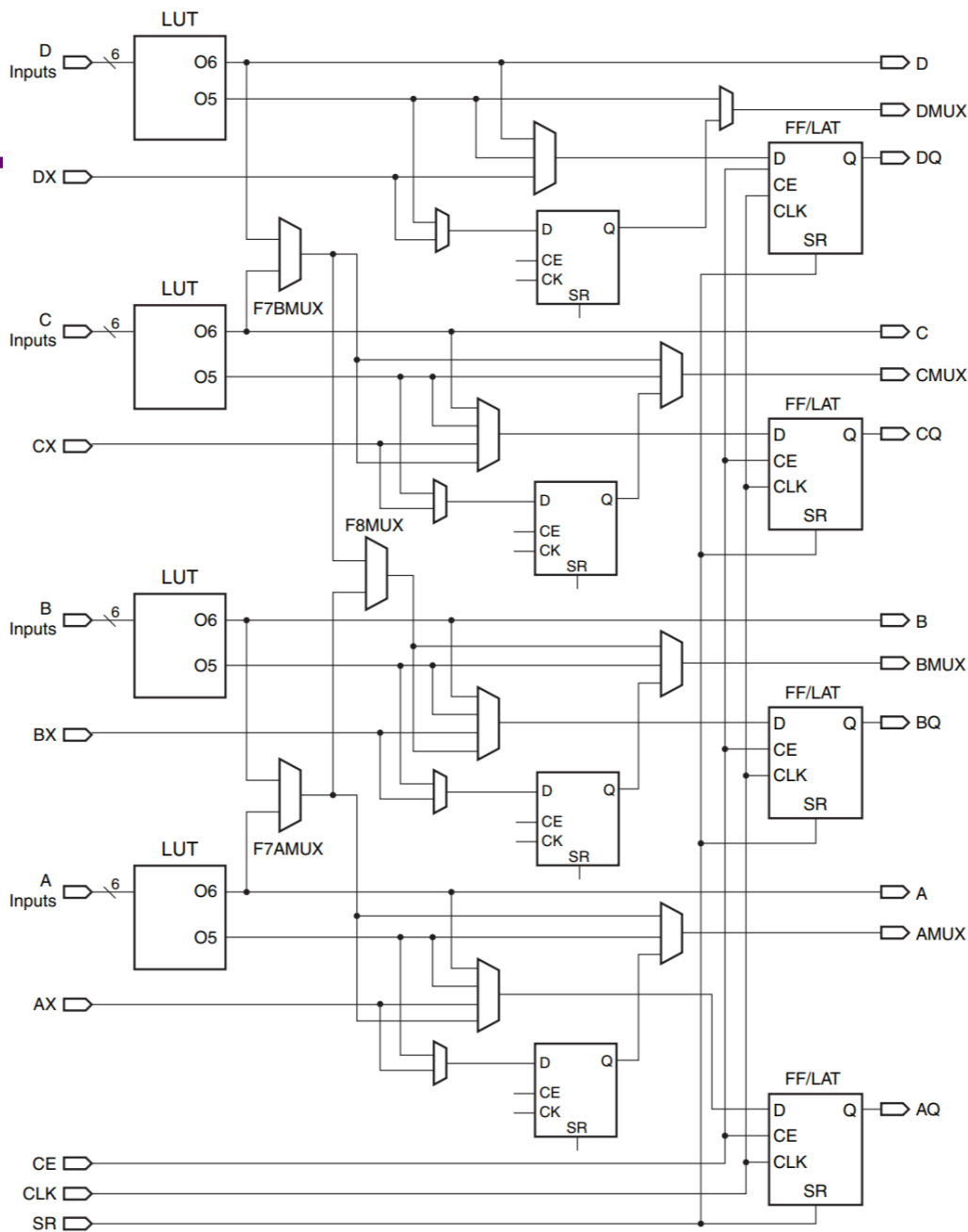


- D是数据输入
- CE是使能信号
- C是时钟信号
- Q是输出
- R是复位信号

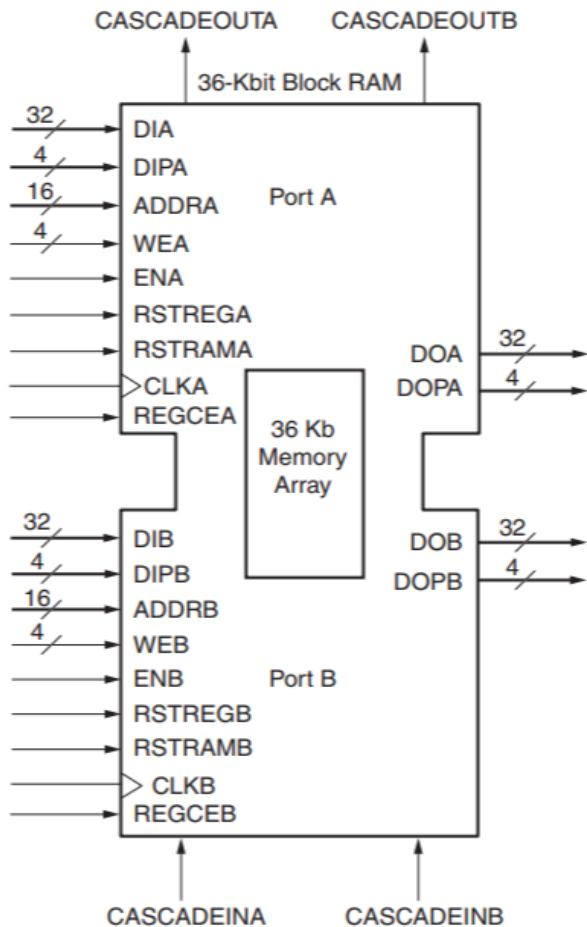
逻辑片SLICE



每两个Slice被组织成一个可配置逻辑块（CLB），最后大量的CLB之间再由开关阵列连接起来。这样的架构使得FPGA片上的LUT、FF可以自由地连接，形成一个更大规模、可配置的逻辑电路

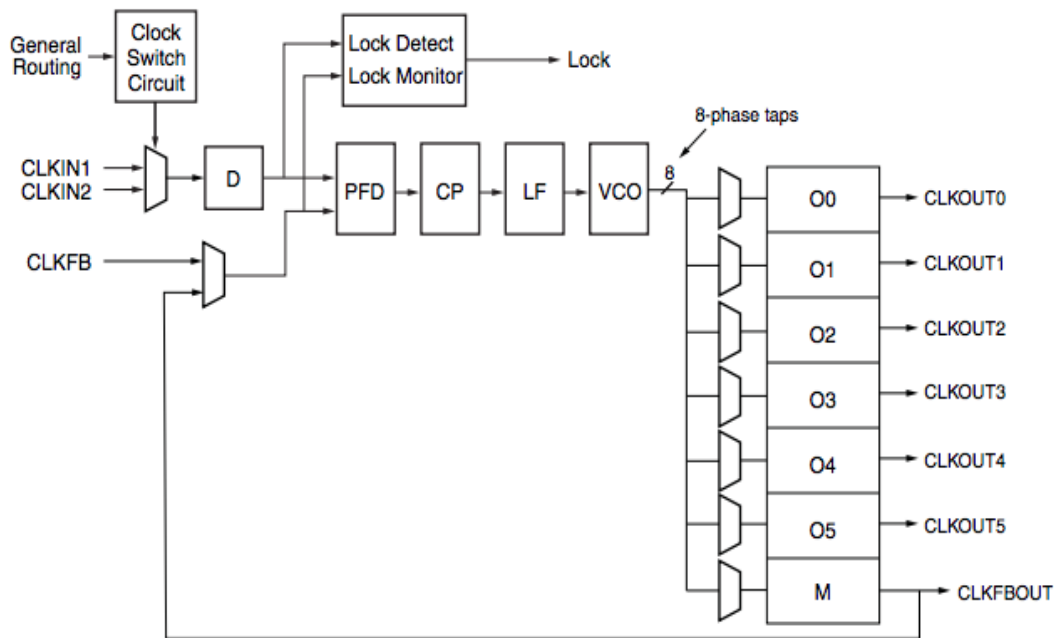


嵌入存储器



- ❑ Block RAM(BRAM)本质上是RAM，是FPGA内部的存储器
- ❑ 共有135个BRAM，总的BRAM容量为4860Kb
- ❑ 使用Block Memory Generator来使用BRAM

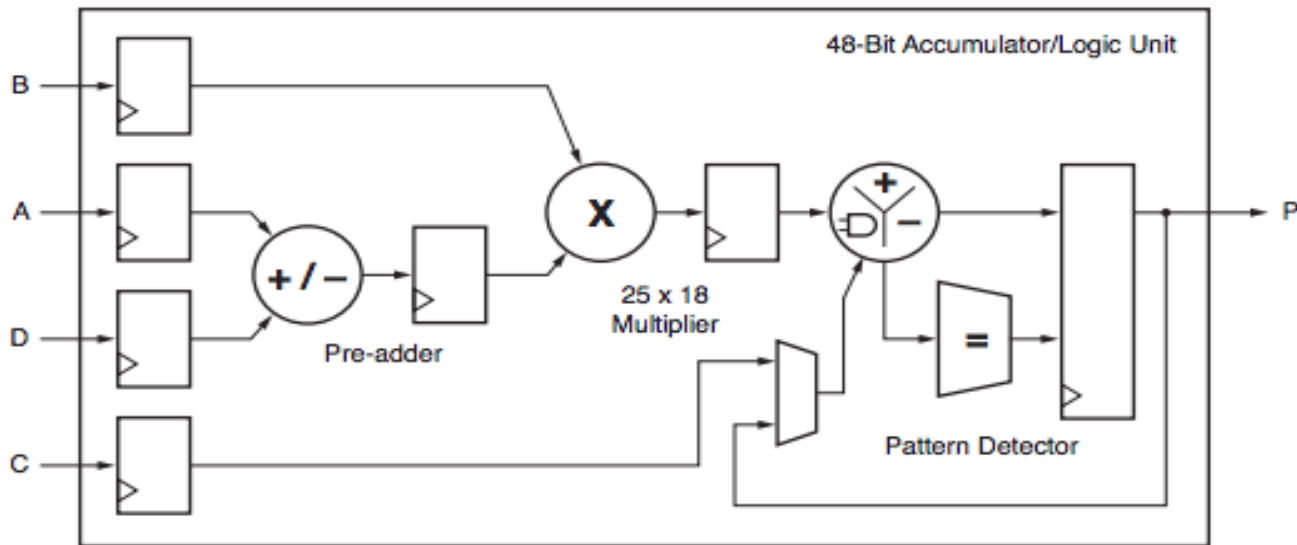
时钟管理单元



❑ 可以对输入的时钟信号进行分频、倍频，从而得到用户需要的时钟

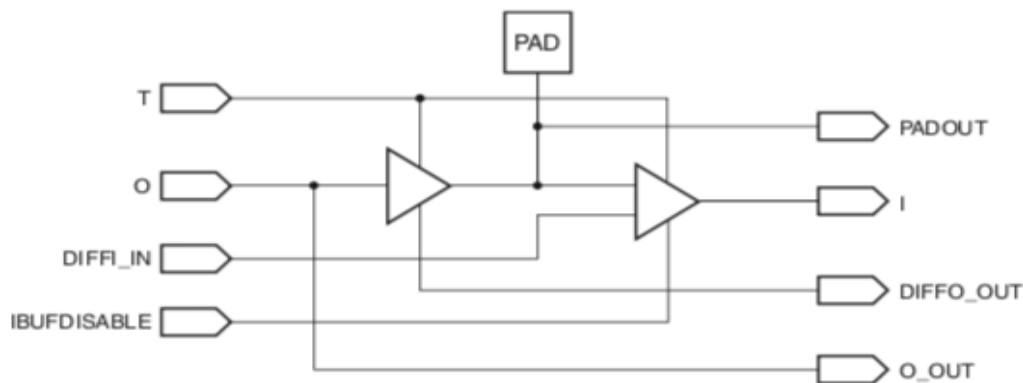
❑ 一个时钟合成器中可以设置多个分频比例，从而得到多个不同频率的时钟

数字信号处理单元



- ❑ 包含一个单周期硬件乘法器和一个加法器（也可以认为是累加器）
- ❑ 可以用来实现乘法指令操作

IO单元



- ❑ I/O口支持输入、输出信号，输出信号还受到3态门控制（可以使得引脚变为高阻态）
- ❑ 由EDA工具根据用户编写的逻辑代码中顶层信号的类型input、output和inout来自动选择

硬件设计的方法与原则

一些硬件设计原则

- 面积和速度的平衡与互换
- 功耗考虑
- 硬件原则
- 系统原则
- 同步设计原则

面积和速度的平衡与互换

- ❑ 面积和速度是数字系统设计考虑的两个重要指标，FPGA作为快速原型设计和系统验证的方法，首先就要考虑到这两个因素直接的平衡问题；
- ❑ 面积指某个FPGA设计综合之后占用的系统资源数，一般用占用的逻辑单元数量及IO接口数量来衡量，这一指标综合软件一般都能给出；
- ❑ 更小的面积通常代表更低的成本。

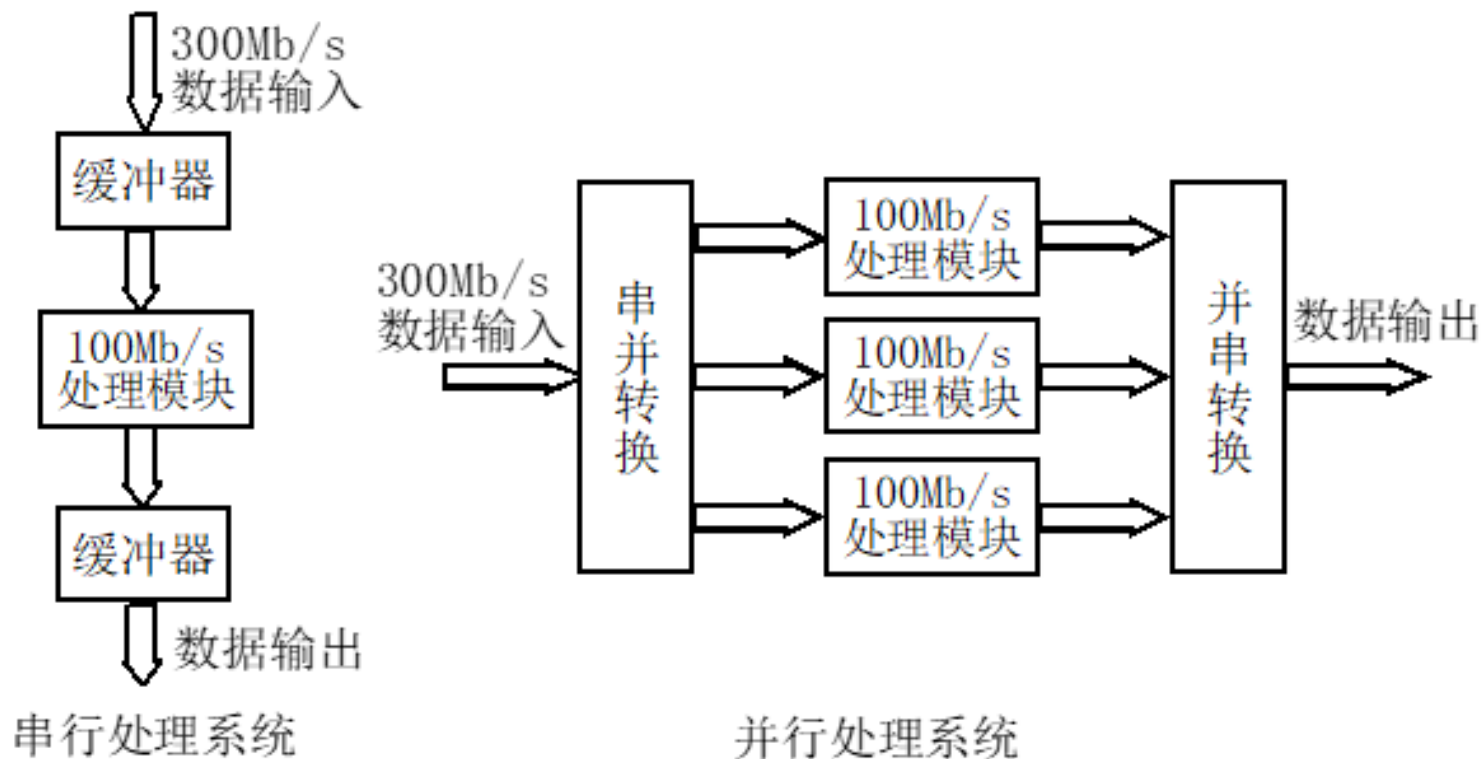
面积和速度的平衡与互换

- ❑ 简单说，速度通常指系统工作的频率，高频率常常代表高速度；
- ❑ 实际上，进行速度优化不仅仅是简单提高频率，而是要仔细考虑系统各个模块在各种工作状态下的时序要求；
- ❑ 另外可以通过并行操作提高速度；
- ❑ 在一定的工艺条件下，面积和速度常常是一对矛盾，因此需要考虑面积和速度的转换问题。

面积和速度的平衡与互换

- ❑ 将原本复用的模块进行复制，变为并行操作的模块，以牺牲面积来换取速度；
- ❑ 很多被复用的模块都是具有逻辑承接或时间先后关系的，无法直接并行化；
- ❑ 需要修改硬件设计，重新对模块做规划。

面积和速度的平衡与互换，例子



速度换面积

- 和利用面积换速度正好相反，把并行模块进行复用，以节省面积；
- 逻辑上更为简单，理论上，只要用足够的存储器，总能把并行的功能相同的模块进行复用；
- 但是要优化好存储器的管理，节省存储器，工作也并不简单。

功耗考虑

□主要考虑动态功耗：

- 动态功耗和逻辑翻转变化的频率成正比；
- 设计时要考虑尽可能不要让所有的单元同时翻转，避免引起过大的功耗；
- 例如，对于状态机的状态分配，之所以采用Gray码，另一个重要原因就是为了降低功耗；

硬件原则

- 用HDL描述硬件进行数字系统，首先应该考虑的是硬件的实现；
- 程序的可读性，必须以硬件实现为前提；
- HDL描述的是硬件的结构和各个模块之间的连接关系，在设计前应对硬件本身有清晰的了解，然后用适当的HDL进行描述；
- 注意避免软件思维 (if, case 等语句不是执行的流程，而是直接被翻译为门电路)
- 多条信号线的信号传输是并行的

系统原则

□ 数字系统设计应该从宏观和系统全局的角度进行考虑；

- 例如对于模块等的复用和合理组织所得到的效果远比对于小部分代码的反复推敲大；

系统原则

□设计前应对所用FPGA的底层硬件资源有所了解：

- 底层可编程硬件单元
 - Xilinx的为Slice
 - Altera的为LE
- Block RAM单元
- 布线资源
- 可配置IO单元
- 时钟资源

同步设计原则

- ❑ 在目前的条件下，采用异步电路设计并不理想，而现在的FPGA芯片都是为同步电路设计优化的
- ❑ 同步电路上也是有设计限制的，例如不能同时响应上升沿和下降沿
- ❑ 单纯从IC设计角度看，同步电路比异步电路更加消耗资源；

同步设计原则

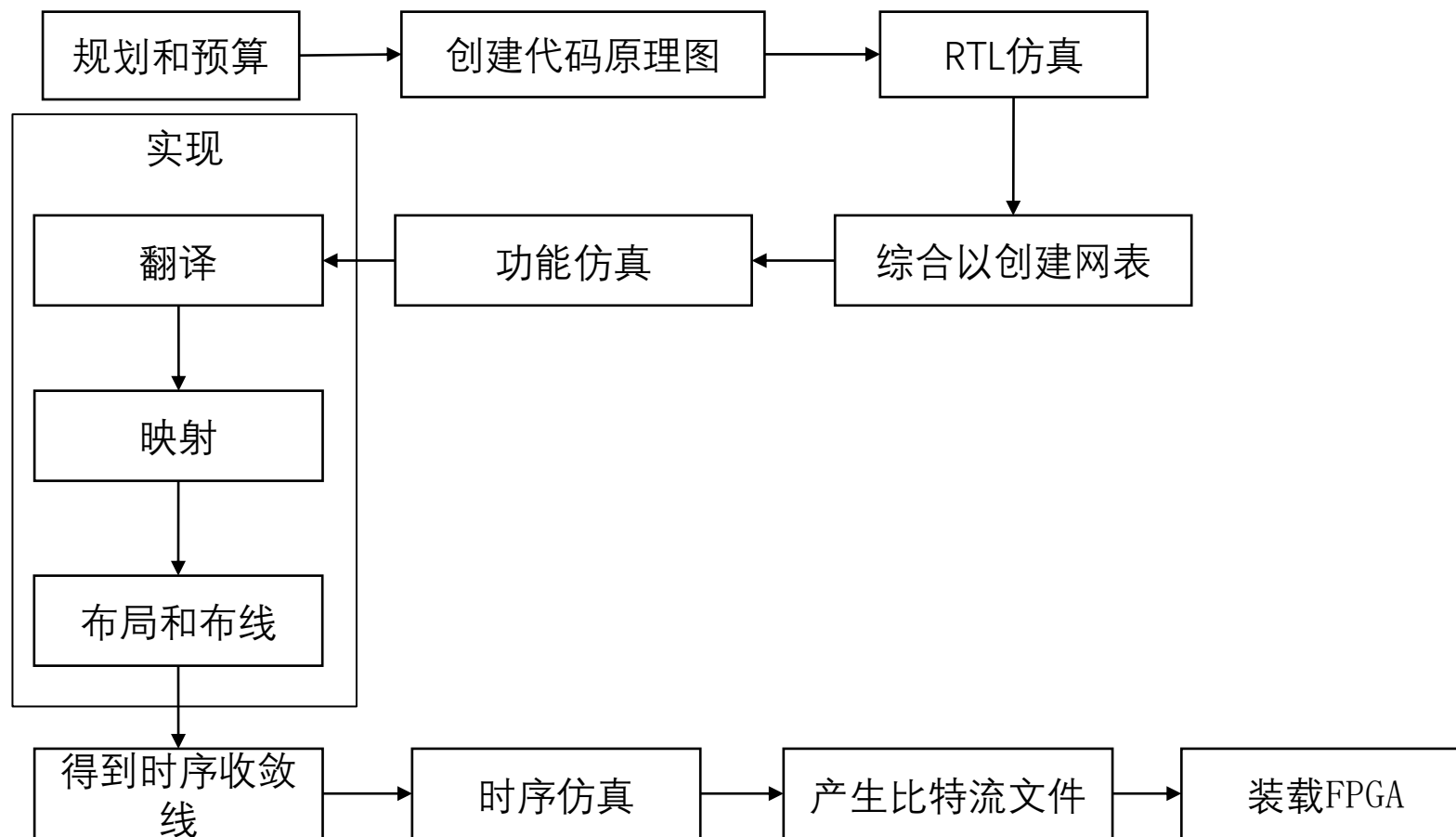
- 从资源考虑，关键要优化两种资源的比例；
- 另外，同步时序电路具有没有毛刺、信号稳定等优点；
- 同步时序电路中延时的产生；
- 同步时序电路中输入的同步；

层次化设计对原理图和HDL都很重要

- 可以将设计概念化
- 将设计结构化
- 使调试设计更容易
- 使设计的不同部分的不同输入设计方法（原理图，HDL，本地编辑）能更容易结合
- 使更容易的更新设计，其中包括设计，实现，以及在设计过程中验证个别元件
- 减少优化时间
- 便于并行设计

硬件开发流程

开发流程



设计流程

- 设计、输入和综合
 - 原理图
 - 硬件描述语言
- 设计实现
 - 生成比特流文件
- 设计验证
 - 门级仿真
 - 下载

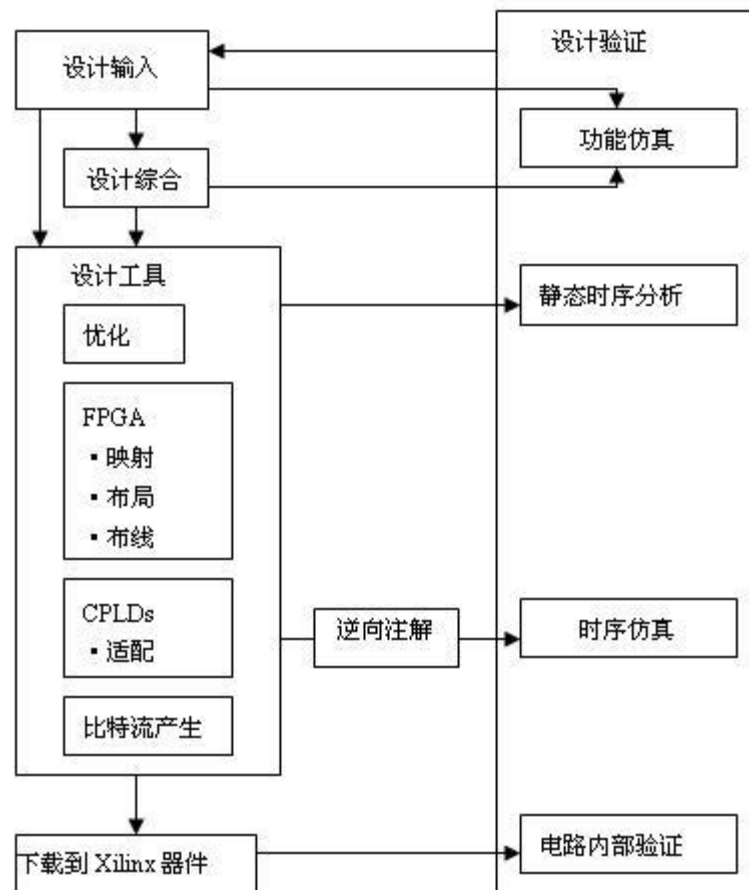


图 1.1 Xilinx 标准的设计流程

设计、输入和综合

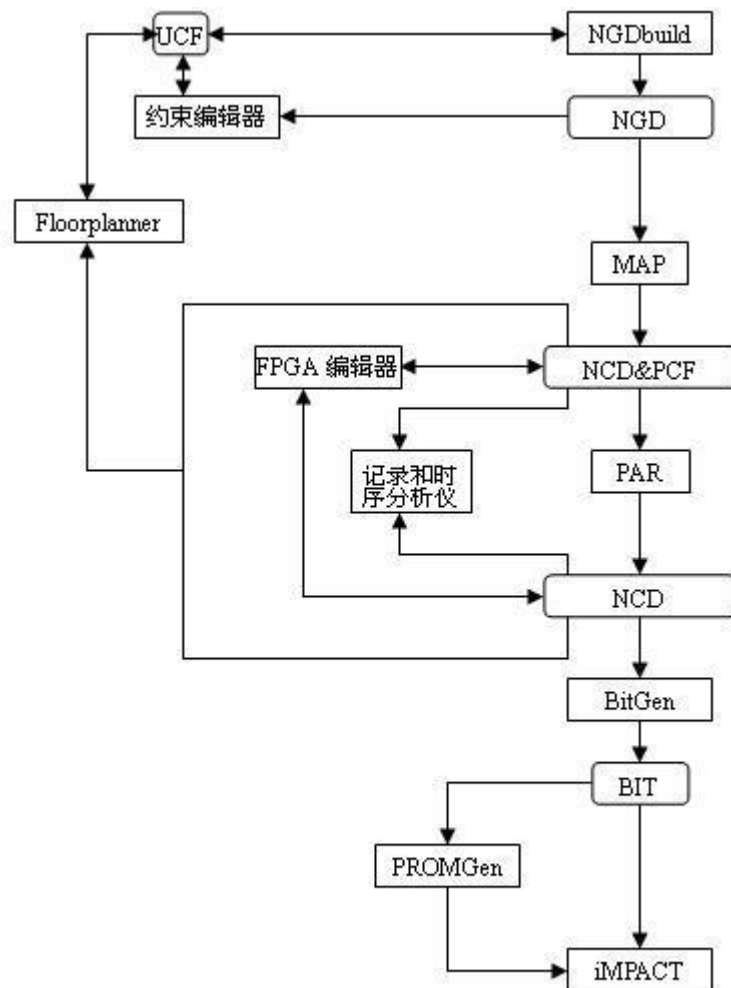
- 方案论证、系统设计和FPGA芯片选择等准备工作
- 输入是将系统或电路以某种形式输入给EDA工具的过程
- 创建设计后可以进行功能仿真，也称为前仿真，是在编译之前对用户所设计的电路进行逻辑功能验证
- 综合就是将较高级抽象层次的描述转化成较低层次的描述

约束

- 如果想要对设计中的时间参数或者布局参数进行约束，设计者可以指定映射、块布局、以及时间规范
- 映射约束
 - 指定逻辑块如何映射到可配置的逻辑块
- 模块布局
 - 块布局可限制在指定位置，可以是多个位置中的其中一个，或者是一个位置区域。
- 时序规范
 - 可以指定设计中路径的时间要求。

设计实现

- 从逻辑设计文件映射或适配到指定的器件开始，到物理设计布线成功并生成比特流文件时结束



设计仿真

□ 综合后仿真

- 把综合生成的标准延时文件反标注到综合仿真模型中去，可估计门延时带来的影响。

□ 时序仿真与验证

- 也称后仿真，是指将布局布线的延时信息反标注到设计网表中检测有无时序违规板级仿真与验证

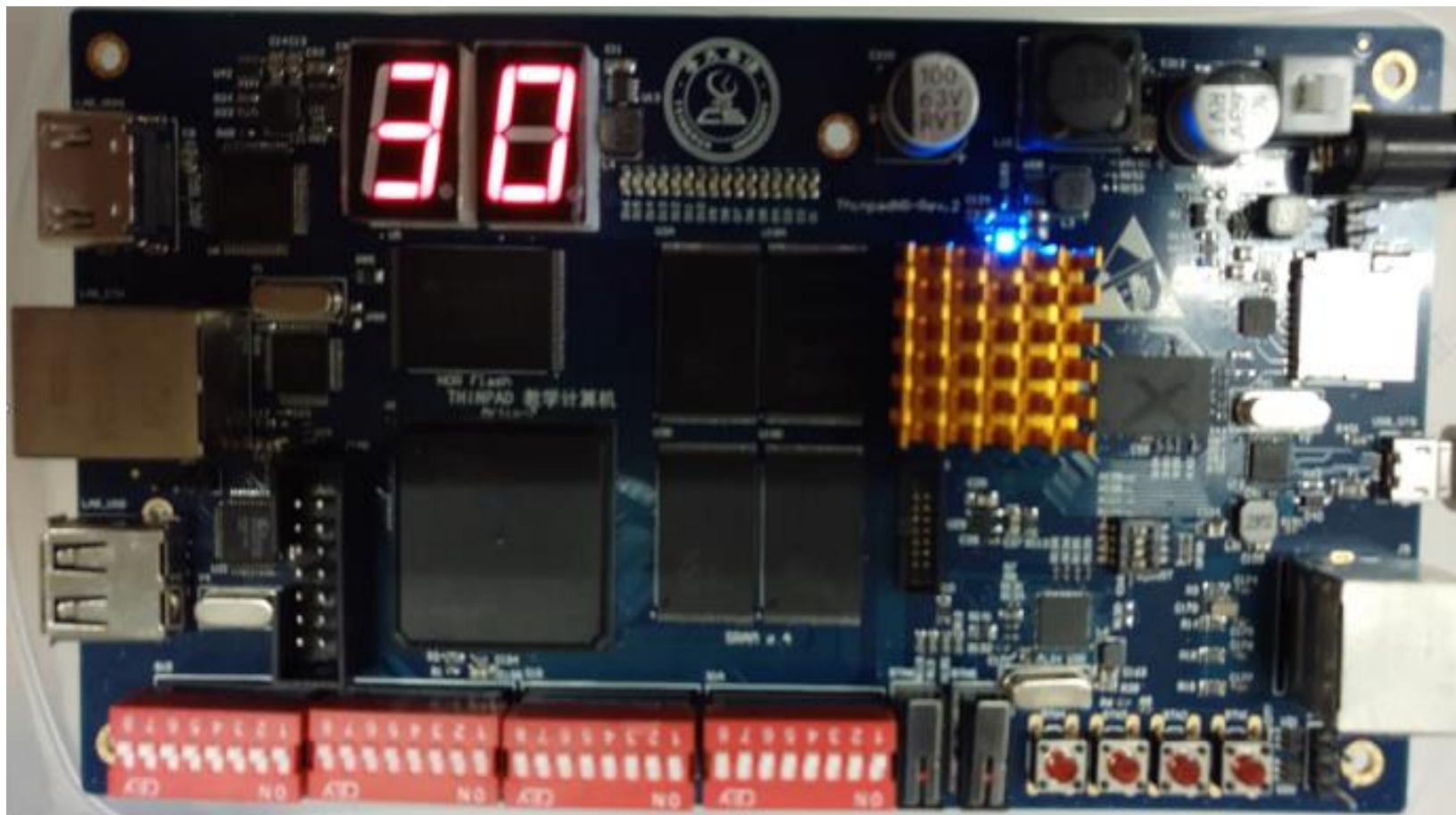
□ 板级仿真

- 主要应用于高速电路设计中，对高速系统的信号完整性、电磁干扰等特征进行分析，一般都以第三方工具进行仿真和验证。

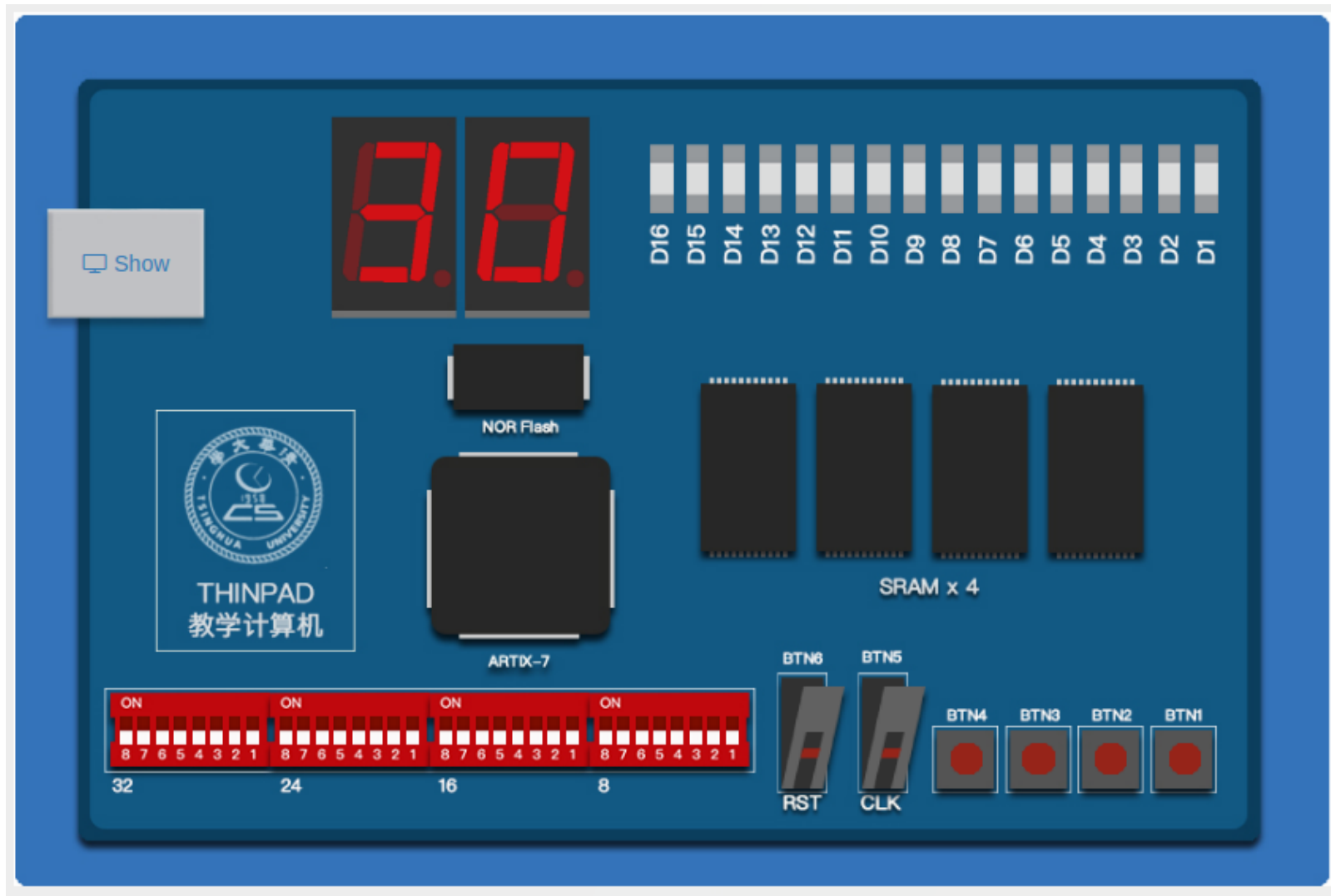
□ 芯片编程与调试

- 将编程数据下载到FPGA芯片中

本地执行



远程执行



参考网址

□ <https://www.fpga4fun.com/>

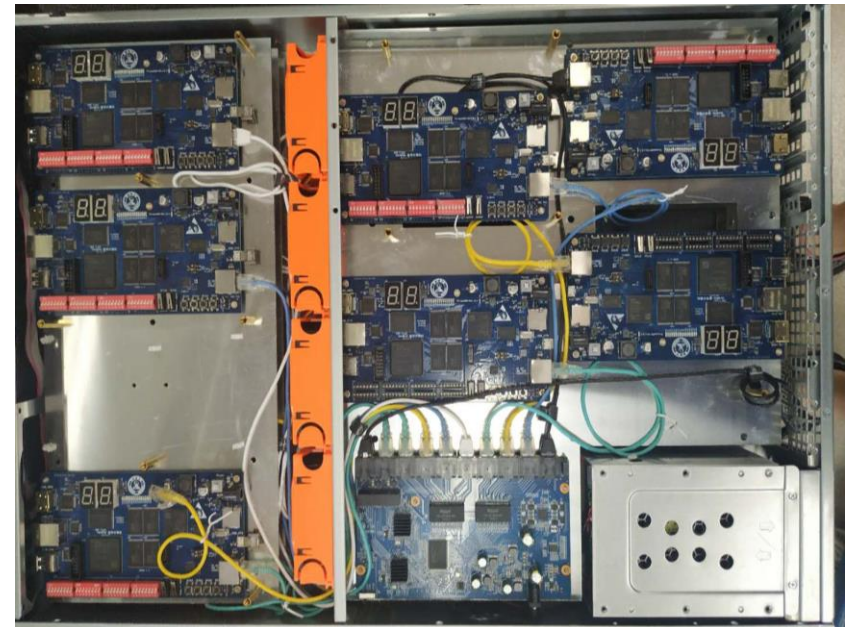
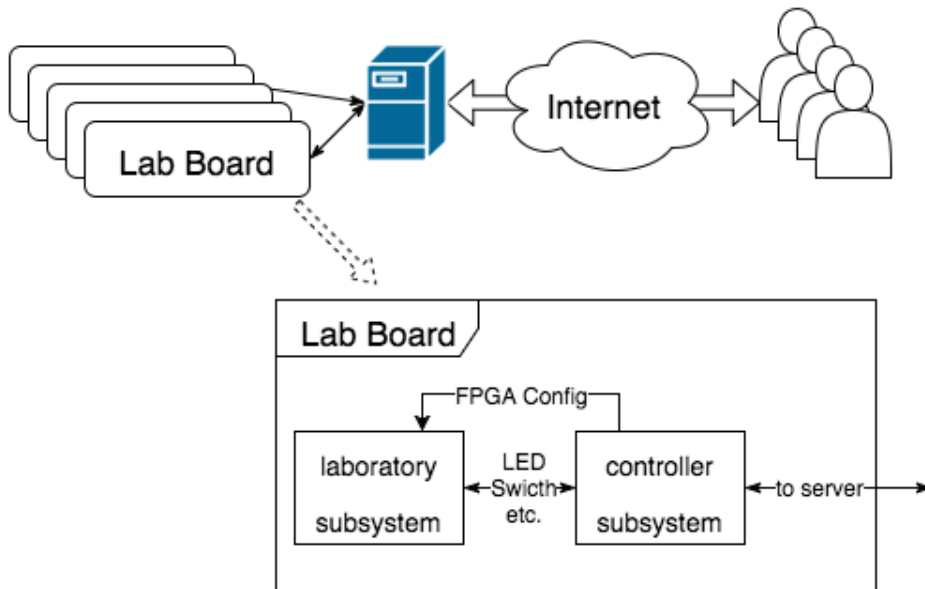
□ 没有学过数字逻辑课程的同学，请务必尽快自学这部分相关的内容，注意下面的基本概念

- 注意组合逻辑的特点
- 注意时序逻辑的特点
- 时钟起到什么作用
- 什么是时延

我们的实验系统

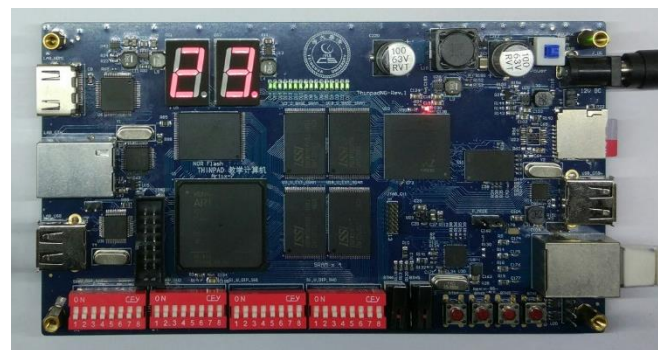
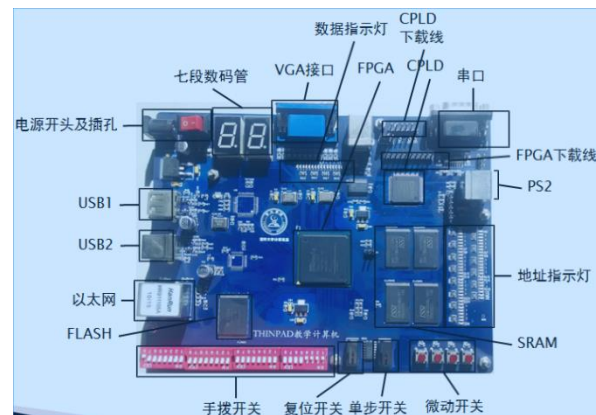
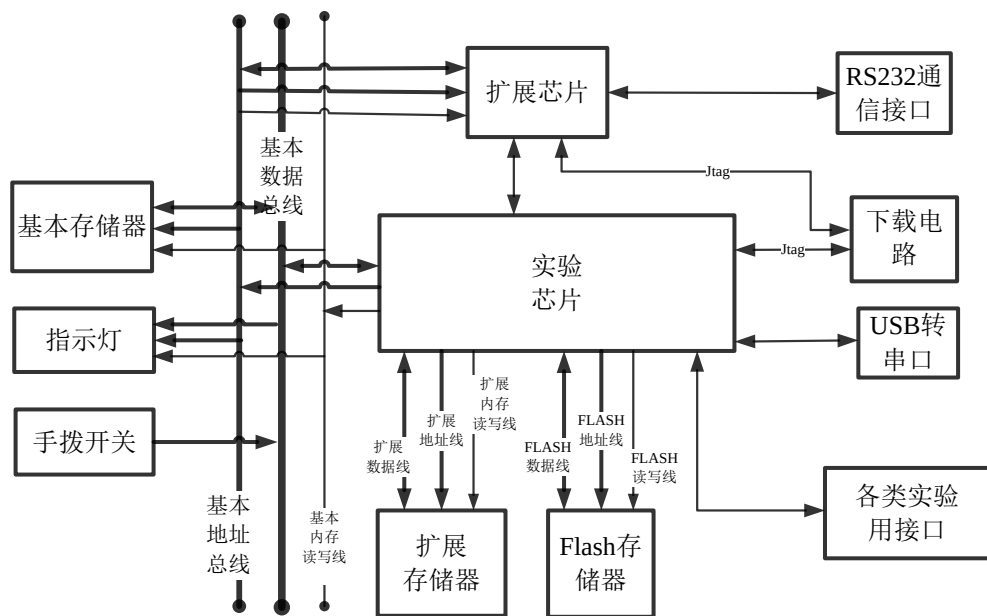
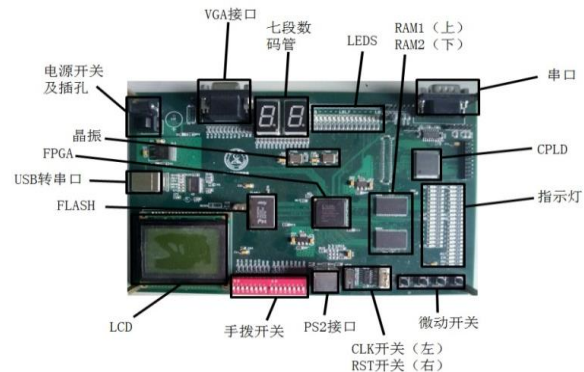
实验系统

□ ThinPAD Cloud

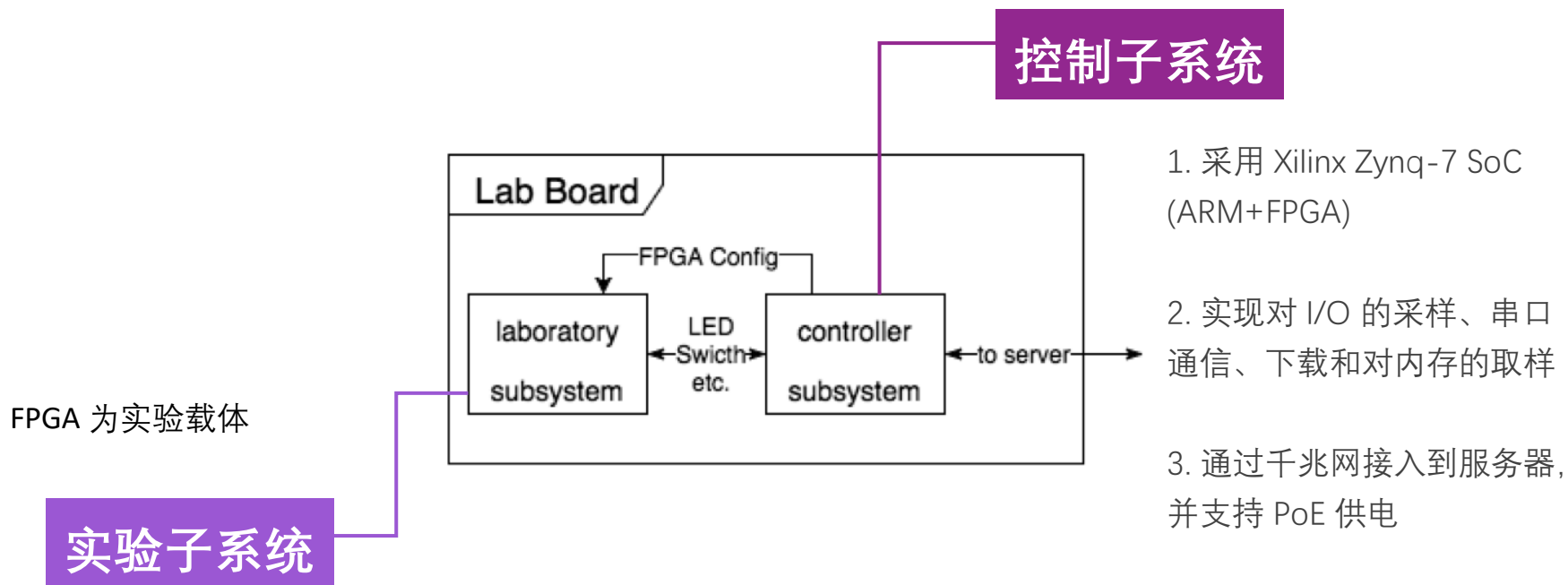


硬件设备

ThinPAD实验设备



系统构架



实验流程

1. 上传设计文件 (bit流)

Upload Your Design

[选择文件](#) soc_toplevel.bit
Choose the .bit file and upload.

Design Name	soc_toplevel;UserID=0XFFFFFFF;Version=2016.3
Build Time	2016/11/06 22:42:14

[Submit](#)

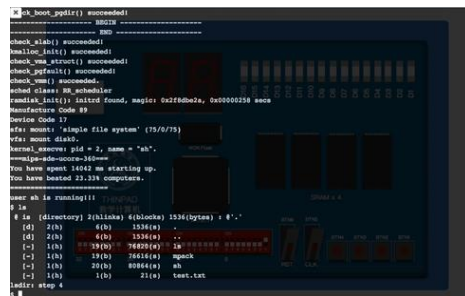
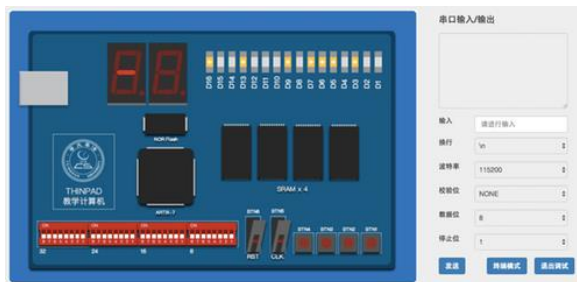
© 2016 Thinpad Plus Group



```
freemem start at: 80088000
free pages: 00000778
## 00000020
check_alloc_page() succeeded!
check_pgdir() succeeded!
check_boot_pgdir() succeeded!
----- BEGIN -----
----- END -----
check_slab() succeeded!
kmaloc_init() succeeded!
check_vma_struct() succeeded!
```

3. 串口读写

2. 在用户界面与硬件交互



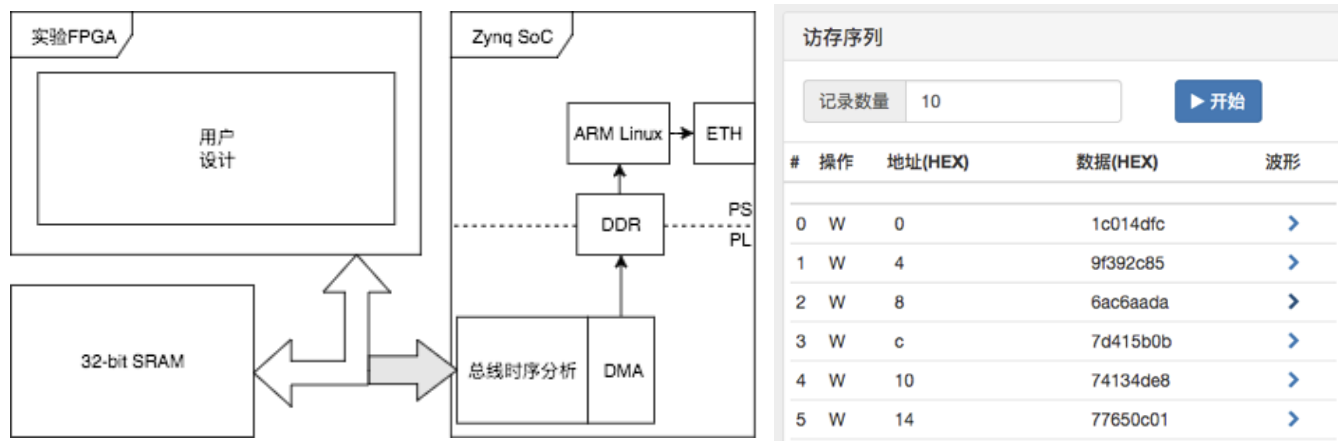
4. 串口终端

VGA终端

在线调试

❑ 硬件在线调试困难，只能用点灯大法？

- 在线ILA
- 内存分析工具
- 软件工具链
 - 模拟器、汇编器、仿真终端、数据通信程序、监控程序、GCC编译器、汇编与链接工具



自动评测

- 单独的评测机制

- 功能评测

- 标准测试样例

- 测试机的影响

- 性能评测

- 频率

- 冲突处理

- 问题：OJ系统

ThinPAD-Cloud 在线实验 自动评测 g102 登出

Computer Organization 40240354

ALU Design

SRAM and UART

MIPS32 CPU

MIPS32 Performance

Challenge

Digital Logic 30240551

NSCSCC Test

题目描述

测试说明

本实验内容为MIPS32 CPU设计实现。测试脚本将利用监控程序（基础版）对CPU功能进行验证，并用一段汇编代码测试每组额外实现的3条扩展指令。测试对CPU实现有如下要求：

1. 内存空间为0x80000000 ~ 0x807FFFFFFF，共8MB。其中0x80000000 ~ 0x803FFFFFFF映射到BaseRAM，0x80400000 ~ 0x807FFFFFFF映射到ExtraRAM。
2. 使用与BaseRAM共享总线的外部串口控制器，地址映射按照监控程序要求。
3. CPU时钟使用外部时钟输入，两路时钟输入均可使用。
4. CPU复位后从0x80000000开始取指令。

选择RISC-V CPU设计的组不提交在线评测，只需要线下检查。

测试步骤

监控程序测试

1. 将拨码开关设置为0
2. Kernel下载到BaseRAM中
3. 单击复位按钮
4. 模拟Tern程序连接串口，等待Kernel启动时的欢迎信息
5. 用A命令将用户程序加载到0x80100000处
6. 用D命令读出用户程序，检查是否正确加载
7. 用G命令执行用户程序
8. 用R和D命令读取用户程序执行后的寄存器和内存，检查是否正确执行

扩展指令测试

1. 将拨码开关设置为0
2. 测试程序加载到BaseRAM中
3. 根据组号，选择待测指令，作为参数写入到0x80100000
4. 单击复位按钮，执行测试程序
5. 测试程序根据参数测试扩展指令
6. 0.5s后从0x80100000处读出测试结果

测试用例列表

ID	名称	说明	内容可见
5dccfbb23aa7d3468edb26e0	Supervisor Operations	supervisor-mips32 functional tests	
5dcf968d3aa7d3468edb2dc2	Extra Instructions	Test 3 extra instructions assigned to each team	

SystemVerilog 的高级 语言特性

Verilog 的四值逻辑

□ 在 Verilog 中，信号共有 4 个状态

- 0, 1, X, Z

□ Z，即高阻态

- 仿真中某个组合逻辑信号没有被赋值时，默认为Z
- 实际电路中除三态门外不能被读取到
- 仿真中会导致问题
- 实际上会变成 0

□ X，即不定态，代指 0/1 不确定

- 实际电路中不存在
- 仿真时，寄存器的初始值
- 仿真中会导致问题，if(X) 和 if(! X) 都是 false
- 运算在能够化简时会化简，但通常会传播 X
- $0 \& X = 0$, $1 \& X = X$

四值逻辑的比较

□ Verilog 中有两套比较运算符

□ ==, !=

- 通常的比较运算
- 只能处理 0 和 1
- 遇到 Z 和 X 时返回 X
- 可以综合

□ ===, !==

- 严格 4 值比较
- 由于 X 实际上不存在, 综合时会变成 == 和!=
- 仿真时使用

case 语句

- ❑ case: 就是 c++ 中的 switch case
- ❑ 右边是一个例子
- ❑ 通常配合 'define 或者 typedef 来描述状态机
- ❑ 组合逻辑记得写好 default

```
1 typedef enum {STATE_0, STATE_1, STATE_2}  
    state_type;  
2 state_type state_r;  
3 always_ff @(posedge clk) begin  
4     if(rst) begin  
5         state_r <= STATE_0;  
6     end else begin  
7         case(state_r)  
8             STATE_0: begin  
9                 state_r <= STATE_1;  
10            end  
11            STATE_1: begin  
12                state_r <= STATE_2;  
13            end  
14            default: begin  
15                ...  
16            end  
17        endcase  
18    end  
19 end  
20
```

四值逻辑中的 case

- 在遇到 X 和 Z 之后情况会变得更加复杂
- case 在 Verilog 中实现成了严格判等
- 有时候需要这种情况
 - 0XXX
 - 10XX
 - 110X
 - 1110
 - 通常是指令编码处理的时候
 - 比如 `case(inst[31:25])` 来判断 RISC-V 的 func7 字段
- 普通的 case 不好处理

解决 case 语句中的四值逻辑

□ casez, casex

- casez 把 Z 当成通配符,
- casex 把 X 和 Z 都通配
- 信号值的 X 和 Z 也会通配
- casex(1X00) 会满足10XZ 也会满足 XZ00
- casez(1X00) 会满足1X0Z 但不会满足 100Z

□ 在这种情况下建议使用

- casez 做通配, 因为一般来说仿真中出现 X 比出现 Z 的概率大
- 需要让 X 的行为 “不正常” 一些

```
1 always_comb begin
2   {int2, int1, int0} = 3'b000;
3   casez (irq)
4     3'b1?? : int2 = 1'b1;
5     3'b?1? : int1 = 1'b1;
6     3'b??1 : int0 = 1'b1;
7     default: {int2, int1, int0} = 3'b000
8   ;
9   endcase
10 end
```

关于循环的讨论

- ❑ SystemVerilog 里面有循环，但是不好用
- ❑ 准确地说是使用场景比较少
- ❑ 一般来说不能用信号值作为循环变量和循环条件
 - 这样做出来的电路很大概率不可综合
- ❑ 复杂的循环通常也不可综合
- ❑ 通常使用 integer 作为循环变量
- ❑ 循环用来复制逻辑

循环的使用：多个寄存器初始化

对多个寄存器初始化，32 个 0 复位的 32 位寄存器

```
1 reg[31:0] registers[0:31];
2 // reg[31:0] registers[32];
3 // same meaning here
4 always_ff @(posedge clk) begin
5     if(rst) begin
6         for(integer i = 0; i < 32; i++) begin
7             registers[i] <= 32'b0;
8         end
9     end else begin
10        ...
11    end
12 end
```

循环的使用：分组运算

□ 进行分组运算操作，四套 8 位加法器

```
1 reg [31:0] a, b, c;  
2 always_ff @(posedge clk) begin  
3     ...  
4     for(integer i = 0; i < 4; i++) begin  
5         a[i*8+7 : i*8] = b[i*8+7 : i*8] + c[i*8+7 : i*8];  
6     end  
7     ^^I// remember: i here acts as a 'constant' for signals  
8 end
```

模板

□ SystemVerilog 的 module 支持静态泛型

- 简化代码编写
- 减少重复代码

```
1 module register_file #(
2     parameter UNIT_SIZE = 8,
3     parameter SIZE = 1024
4 )(
5     ...
6 );
7 reg[UNIT_SIZE - 1 : 0] mem[SIZE];
8 endmodule
```

```
1 register_file #(
2     .UNIT_SIZE(16),
3     .SIZE(64)
4 ) rf (
5     ...
6 );
```

一些有用的宏: \$clog

□ \$clog2(x)

- 取以 2 为底的对数，上取整
- x 必须是常数
- 通常用于计算信号位宽

□ 比如希望输入一个raddr，然后返回其中的内容

```
1 module register_file #(
2     parameter UNIT_SIZE = 8,
3     parameter SIZE = 1024
4 )
5     input  wire [$clog2(SIZE) - 1 : 0] raddr_i,
6     output wire [UNIT_SIZE - 1 : 0]   rdata_o
7 );
8     reg[UNIT_SIZE - 1 : 0] mem[SIZE];
9 endmodule
10
```

一些有用的宏：\$signed

□ \$signed: 用于强制综合器以有符号方式解读信号数据

```
1 reg[31:0] a, b, c;  
2 always_comb begin  
3     b = $signed(a) >>> 5;  
4     // arithmetic right shift  
5     c = a >>> 5;  
6     // logical right shift  
7     // c = a >> 5  
8 end
```

函数

- ❑ function 用来抽象重复组合逻辑，防止编写重复代码
- ❑ 提供方法来让wire使用 if 等高级特性

```
1 function [31:0] And;  
2 input wire[31:0] a, b;  
3 begin  
4     // you can use if / case / for / ... here  
5     if(a == 32'b0 || b == 32'b0)  
6         And = 32'b0;  
7     else  
8         And = a & b;  
9 end  
10 endfunction;  
11 wire [31:0] a, b, c;  
12 assign c = And(a, b);
```

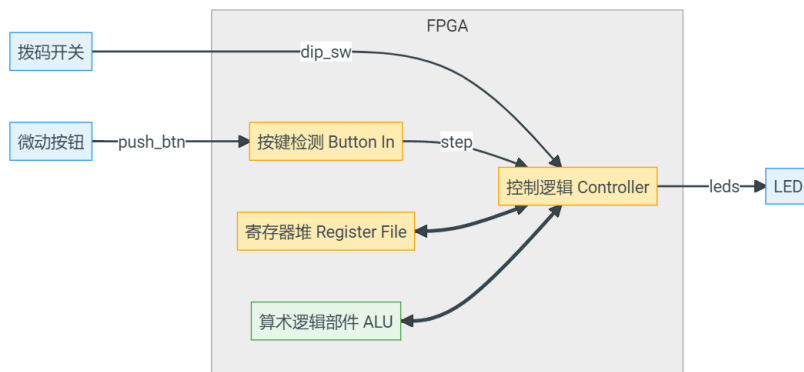
Lab2指导

实验二

□ 实现完整的 ALU 和寄存器堆模块，并通过一个控制状态机对二者进行控制，成为一个可以执行简单运算指令，并可以输入、输出数据的计算器

□ 设计需求

- 使用 32 位拨码开关和 push_btn 按键控制计算逻辑的工作。每次在拨码开关上输入一条 32 位的指令，按下 push_btn，计算逻辑自动控制，完成指令中给定的操作，包括从寄存器中取出两个操作数进行计算、将立即数装载到寄存器，和从寄存器中读出操作数显示在 LED 上三种。

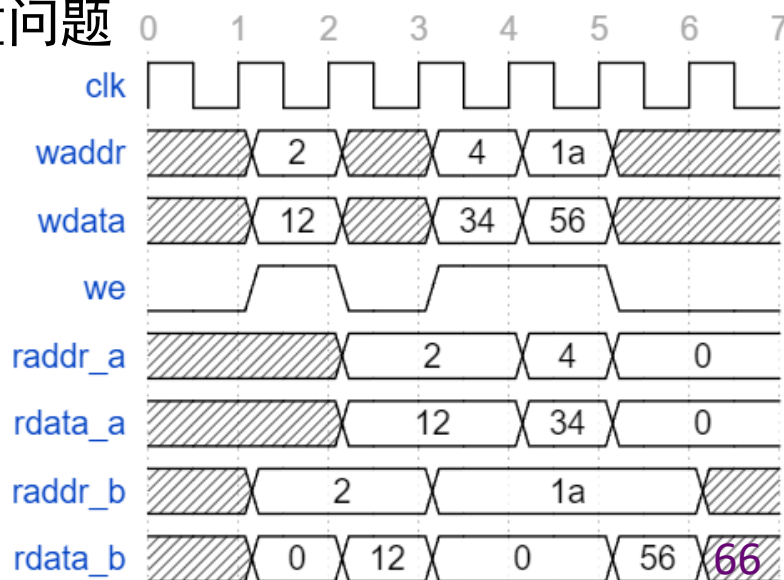


功能

- ❑ 拨码开关读取指令
- ❑ 根据指令完成计算/写寄存器/输出到 led
- ❑ 使用状态机完成任务
- ❑ 状态机转移由按键开关控制

寄存器堆

- 定义和初始化已经给出
- 剩余的就是读写逻辑
 - 信号可以作为信号组下标直接使用
 - 读: `assign rf_rdata_a = registers[rf_raddr_a];`
 - 写: `if(rf_we) registers[rf_waddr] <= rf_wdata;`
- 建议用模板规定寄存器位宽和数量
 - 可以在后续实验中直接复用
 - 仓库中出现同名 module 会导致严重问题



控制逻辑本质是一个有限状态

□ INIT

- 功能：转移时读入指令
- 转移：步进时转移至 decode，其余时刻保持 INIT

□ DECODE

- 功能：解码指令，生成对应信号，并寄存
- 转移：步进时，根据指令转移到 READ_REG, WRITE_REG, CALC

□ READ_REG

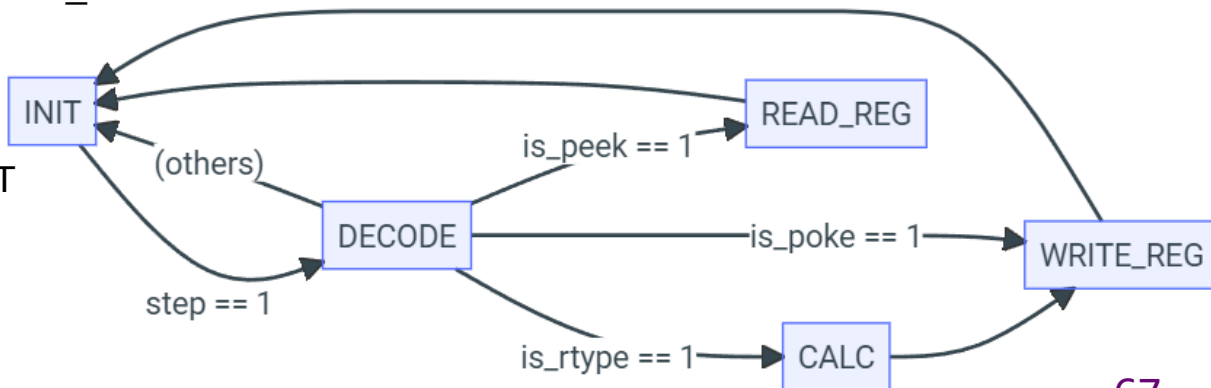
- 功能：读寄存器，生成输出信号，并寄存
- 转移：步进时，转移至 INIT

□ CALC

- 利用 ALU 计算，寄存计算结果
- 转移：步进时，转移至 WRITE_REG

□ WRITE_REG

- 功能：根据值写入寄存器
- 转移：步进时，转移到 INIT



状态机

- 请牢记这个状态设计，后续课程会用到
 - 这是执行一条指令使用的通用执行模式
 - 读指令，译码，计算，访存/输出，写寄存器
- 试着将提供的模板代码改成 3 段式的写法

仿真

□ 生成指令的宏已经写好了，可以直接用

□ 不好的做法

- 暴力调用宏生成指令
- 暴力生成按键和指令序列

□ 好的做法

- 写一个 task 来发送指令数据，控制按键开关
- 在仿真里面直接写一个对拍然后直接 assert

记得看warning 查latch!!

```
module lab3_tb;

    wire clk_50M, clk_11M0592;

    reg push_btn;    // BTN5 按钮开关，带消抖电路，按下时为 1
    reg reset_btn;   // BTN6 复位按钮，带消抖电路，按下时为 1

    reg [3:0] touch_btn; // BTN1~BTN4，按钮开关，按下时为 1
    reg [31:0] dip_sw;   // 32 位拨码开关，拨到“ON”时为 1

    wire [15:0] leds;    // 16 位 LED，输出时 1 点亮
    wire [7:0] dpy0;     // 数码管低位信号，包括小数点，输出 1 点亮
    wire [7:0] dpy1;     // 数码管高位信号，包括小数点，输出 1 点亮

    // 实验 3 用到的指令格式
    `define inst_rtype(rd, rs1, rs2, op) \
        {7'b0, rs2, rs1, 3'b0, rd, op, 3'b001}

    `define inst_itype(rd, imm, op) \
        {imm, 4'b0, rd, op, 3'b010}

    `define inst_poke(rd, imm) `inst_itype(rd, imm, 4'b0001)
    `define inst_peek(rd, imm) `inst_itype(rd, imm, 4'b0010)

    logic [3:0] opcode;
```

```
initial begin
    // 在这里可以自定义测试输入序列，例如：
    dip_sw = 32'h0;
    touch_btn = 0;
    reset_btn = 0;
    push_btn = 0;

    #100;
    reset_btn = 1;
    #100;
    reset_btn = 0;
    #1000; // 等待复位结束

    // 样例：使用 POKE 指令为寄存器赋随机初值
    for (int i = 1; i < 32; i = i + 1) begin
        #100;
        rd = i;    // only lower 5 bits
        dip_sw = `inst_poke(rd, $urandom_range(0, 65536));
        push_btn = 1;

        #100;
        push_btn = 0;

        #1000;
    end

    // TODO: 随机测试各种指令
    #10000 $finish;
end
```

仿真

内容概要

□ 仿真简介

□ SystemVerilog 仿真语言

□ Testbench 编写

□ Vivado 仿真操作

Verilog 项目都是仿真驱动开发的。

—— 张宇翔

电路即黑盒

□ 所有的电路都可以抽象为黑盒

- 每个黑盒都有输入和输出信号
- 黑盒根据不同的输入信号，会有不同的行为
- 行为结果反映在内部状态和对外输出上
- 回想一下设计电路实现状态机时候用的方法
- 当前状态/输入 $\Rightarrow$ 输出/新状态
- 所有的同步时序电路都可以抽象成这样

黑盒测试

- 电路上到 FPGA 上之后很难获得运行时信息
- 电路的调试依赖上板前的检查
 - 读代码
 - 仿真
- 既然电路已经被抽象为黑盒，需要给黑盒提供对应的输入，并调用黑盒的初始化方法，之后观察黑盒的输出

回顾软件的设计过程

□ 编写程序

- IDE、Linter 查错 ==> 基本的语法正确性

□ 编译程序

- 编译器报错、警告 ==> 语法、简单的错误检查

□ 运行测试

- 控制台输出调试 ==> 获得程序状态
- GDB 等调试工具 ==> 深层次的运行时检查
- 设计单元测试等 ==> 详细覆盖各种可能情况

硬件设计与调试过程

□ 编写程序

- IDE、Linter 查错 => 基本的语法正确性

□ 编译程序

- 综合器报错、警告 => 语法、简单的错误检查

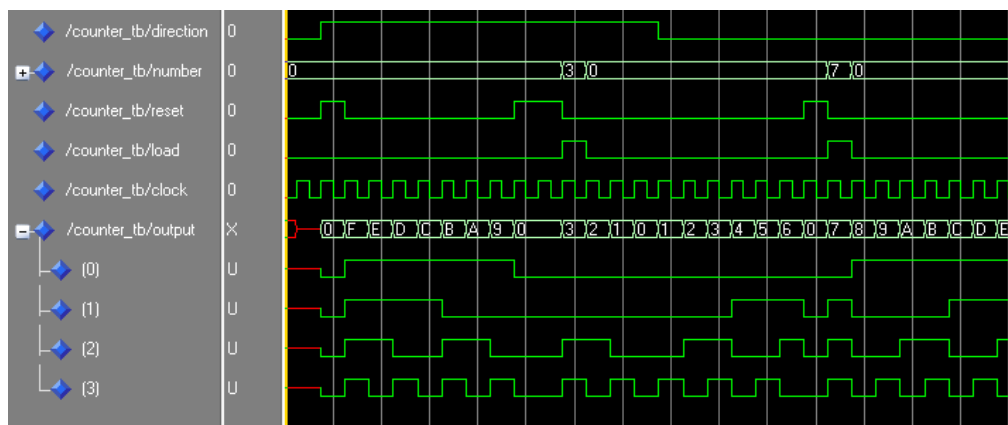
□ 运行测试

- HDL 描述的逻辑在芯片内，难以获取内部状态
 - 编写的 HDL 与需求是否一致？
 - 逻辑输出不正确，为什么？
- 硬件上运行测试难以覆盖各种输入情况

数字设计中的仿真

□ 仿真的作用

- 大幅降低硬件设计的调试难度
 - 无需综合，快速了解设计的基本正确性
 - 便利地查看内部信号的实时状态
- 提高输入情况的覆盖效率
 - 高层次语言设计、随机化输入输出
 - 覆盖率统计、形式化验证（本课程不涉及）



Vivado中的仿真

□ 行为仿真

- 写完设计源文件、tb文件之后，不综合，直接进行仿真，只考虑了HDL描述的功能是否符合设计要求，不考虑电路门延迟和线延迟。
- 可以用来检查代码中的语法错误以及代码行为的正确性，其中不包括延时信息。如果没有实例化一些与器件相关的特殊底层元件的话，这个阶段的仿真也可以做到与器件无关。因此在设计的初期阶段不使用特殊底层元件即可以提高代码的可读性、可维护性，又可以提高仿真效率，且容易被重用。

Vivado中的仿真

- 功能仿真
- 综合或实现之后进行的仿真。
- 综合工具除了可以输出一个标准网表文件以外，还可以输出Verilog或者VHDL网表，其中标准网表文件是用来在各个工具之间传递设计数据的，并不能用来做仿真使用，而输出的Verilog或者VHDL网表可以用来仿真。

Vivado中的仿真

□ 时序仿真

□ 综合的过程，将设计输入编译成由与、或、非门，RAM，触发器等基本逻辑单元组成的逻辑连接，即网表（Netlist），并输出edf、edn等标准格式的网表文件。综合后仿真把综合生成的标准延时文件反标注到综合仿真模型中去，可估计门延时对电路带来的影响。

□ 实现与布线，根据所选芯片的型号，将综合输出的逻辑网表适配到具体的FPGA/CPLD上。实现过程中最主要的过程是布局布线（Place and Route）：布局将逻辑单元合理地适配到FPGA内部的固有硬件结构上；布线则根据布局的拓扑结构，利用FPGA内部的各种连线资源，合理正确地连接各个元件。时序仿真将布局布线的延时信息反标注到设计网表中进行仿真。此时的仿真延时文件信息最全，包含门延时和布线延时，所以布线后仿真最准确，能较好地反映芯片的实际工作情况。

基本概念

□ DUT (Device Under Test)

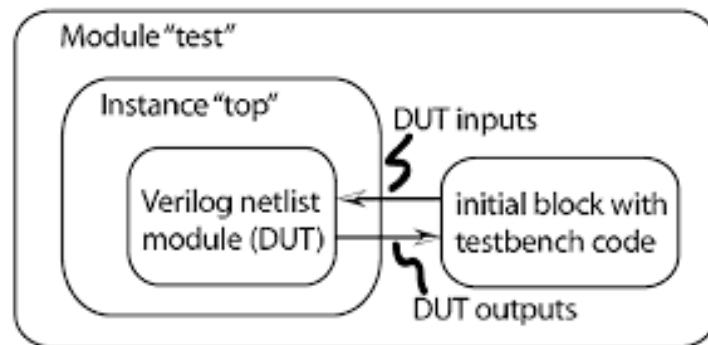
- 待测的设计模块
- 可以是完整的设计，如 thinpad_top 模块
- 也可以是设计中的单个模块，如 regfile

□ Testbench

- SystemVerilog 编写
- 连接信号线

□ 其他模块

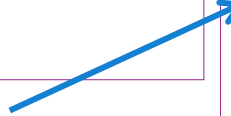
- 时钟生成
- 激励信号生成
- 时序规则检测等



代码样例

```
module halfAdd (sum, cOut, a, b);  
    output sum, cOut;  
    input a, b;  
    xor (sum, a, b);  
    and (cOut, a, b);  
endmodule
```

待测模块 DUT



```
module tb;  
    logic a, b, sum, cOut;
```

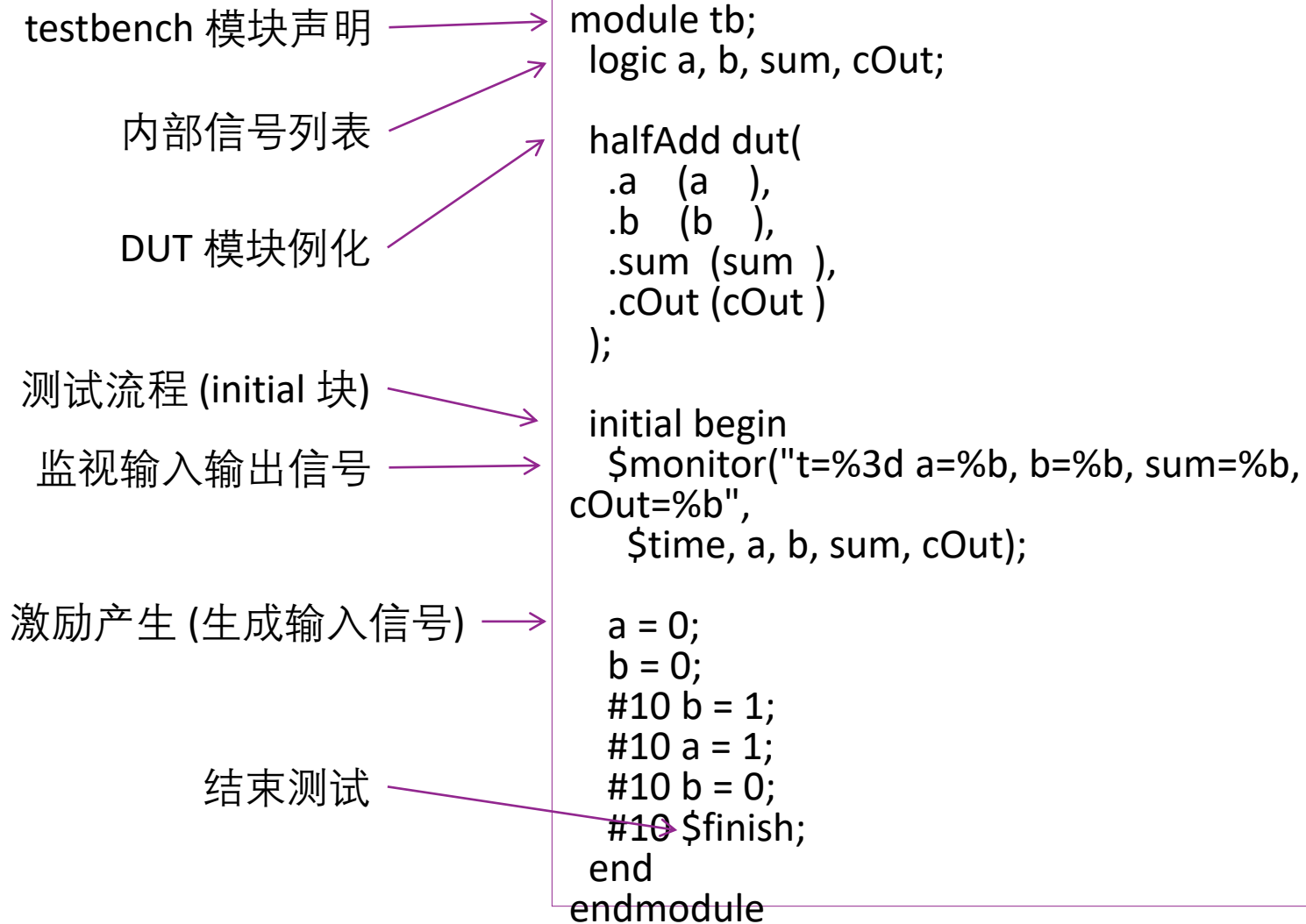
```
    halfAdd dut(  
        .a (a ),  
        .b (b ),  
        .sum (sum ),  
        .cOut (cOut )  
    );
```

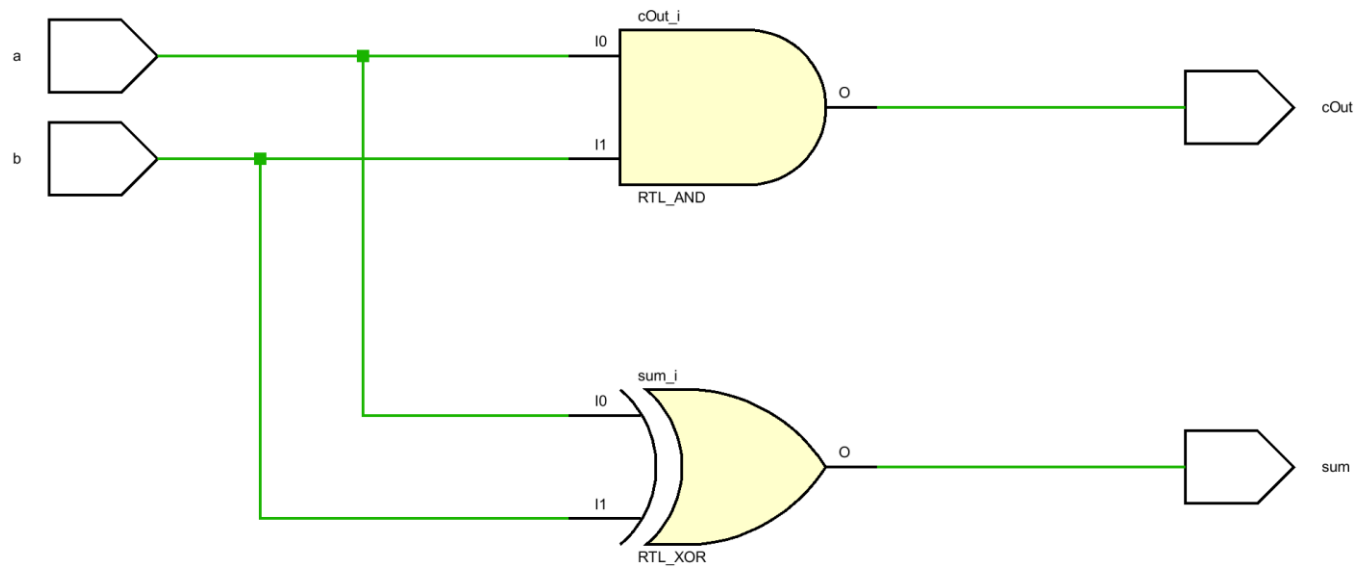
```
    initial begin  
        $monitor("t=%3d a=%b, b=%b, sum=%b,  
cOut=%b",  
            $time, a, b, sum, cOut);
```

```
        a = 0;  
        b = 0;  
        #10 b = 1;  
        #10 a = 1;  
        #10 b = 0;  
        #10 $finish;  
    end  
endmodule
```

测试平台 Testbench

Testbench 结构





▼ SIMULATION

Run Simulation

▼ RTL ANALYSIS

► Open Elaborate Model

▼ SYNTHESIS

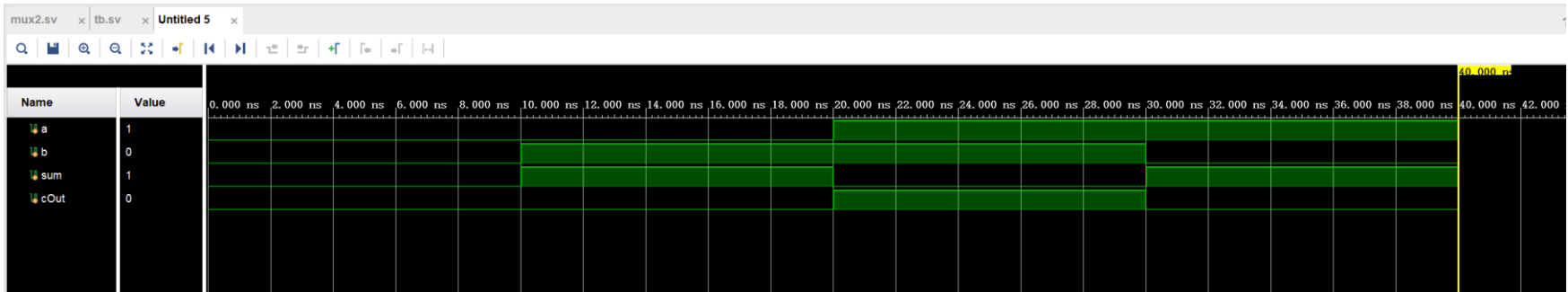
Run Behavioral Simulation

Run Post-Synthesis Functional Simulation

Run Post-Synthesis Timing Simulation

Run Post-Implementation Functional Simulation

Run Post-Implementation Timing Simulation

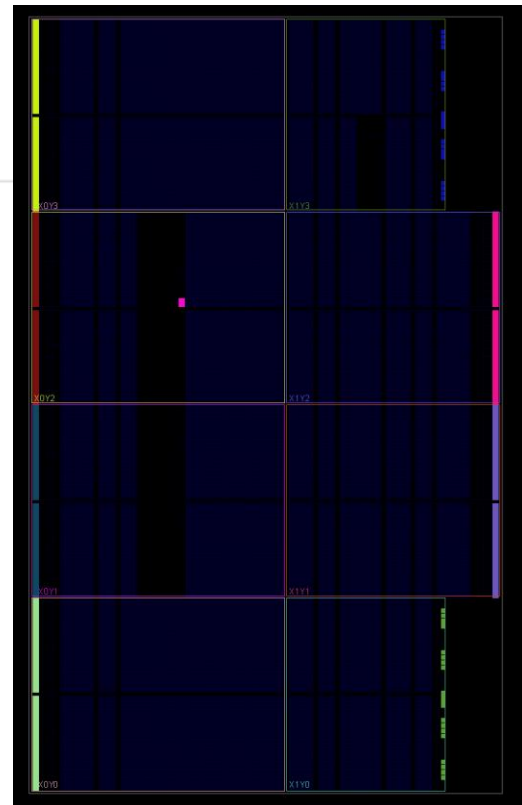
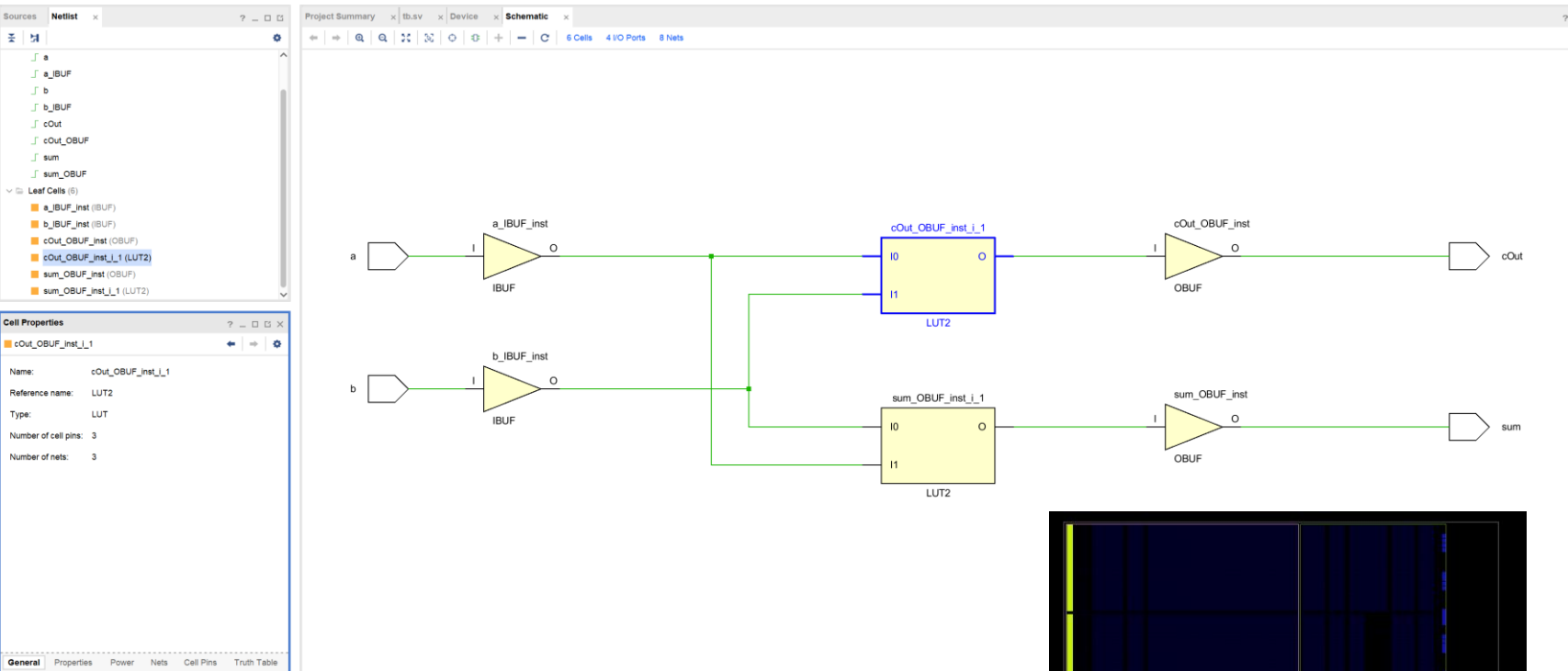


Tcl Console

Messages

Log

```
# add_wave /
# set_property needs_save false [current_wave_config]
# } else {
#   send_msg_id Add_Wave-1 WARNING "No top level signals found. Simulator will start without a wave window. If you want to open a wave window go to 'File->New Waveform'"
# }
# }
# run 1000ns
t= 0 a=0, b=0, sum=0, cOut=0
t= 10 a=0, b=1, sum=1, cOut=0
t= 20 a=1, b=1, sum=0, cOut=1
t= 30 a=1, b=0, sum=1, cOut=0
$finish called at time : 40 ns : File "G:/testprj/testprj.srscs/sources_1/new/tb.sv" Line 42
INFO: [USF-XSim-96] XSim completed. Design snapshot 'tb_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 1000ns
launch_simulation: Time (s): cpu = 00:00:04 ; elapsed = 00:00:12 . Memory (MB): peak = 3368.125 ; gain = 0.000
```



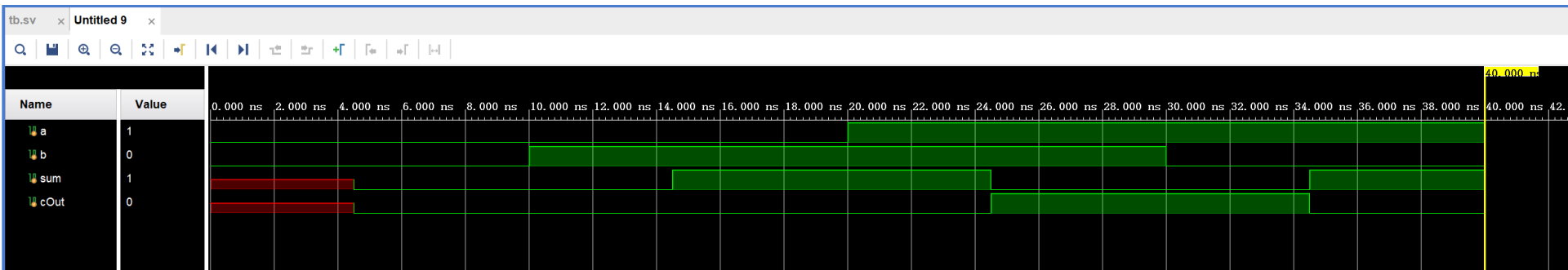
[Run Simulation](#)

> Open E

SYNTHESIS

Run Simulation

Run Post-Implementation Timing Simulation



Sources Netlist x ? _ □ □

halfAdd

- Nets (8)
- Leaf Cells (6)
 - a_IBUF_inst (IBUF)
 - b_IBUF_inst (IBUF)
 - cOut_OBUF_inst (OBUF)
 - cOut_OBUF_inst_i_1 (LUT2)
 - sum_OBUF_inst (OBUF)
 - sum_OBUF_inst_i_1 (LUT2)

Cell Properties ? _ □ □ x

cOut_OBUF_inst_i_1

Name: cOut_OBUF_inst_i_1

Reference name: LUT2

Type: LUT

BEL: ☒ ASLUT ☐ Fixed

Site: ☒ SLICE_X0Y1

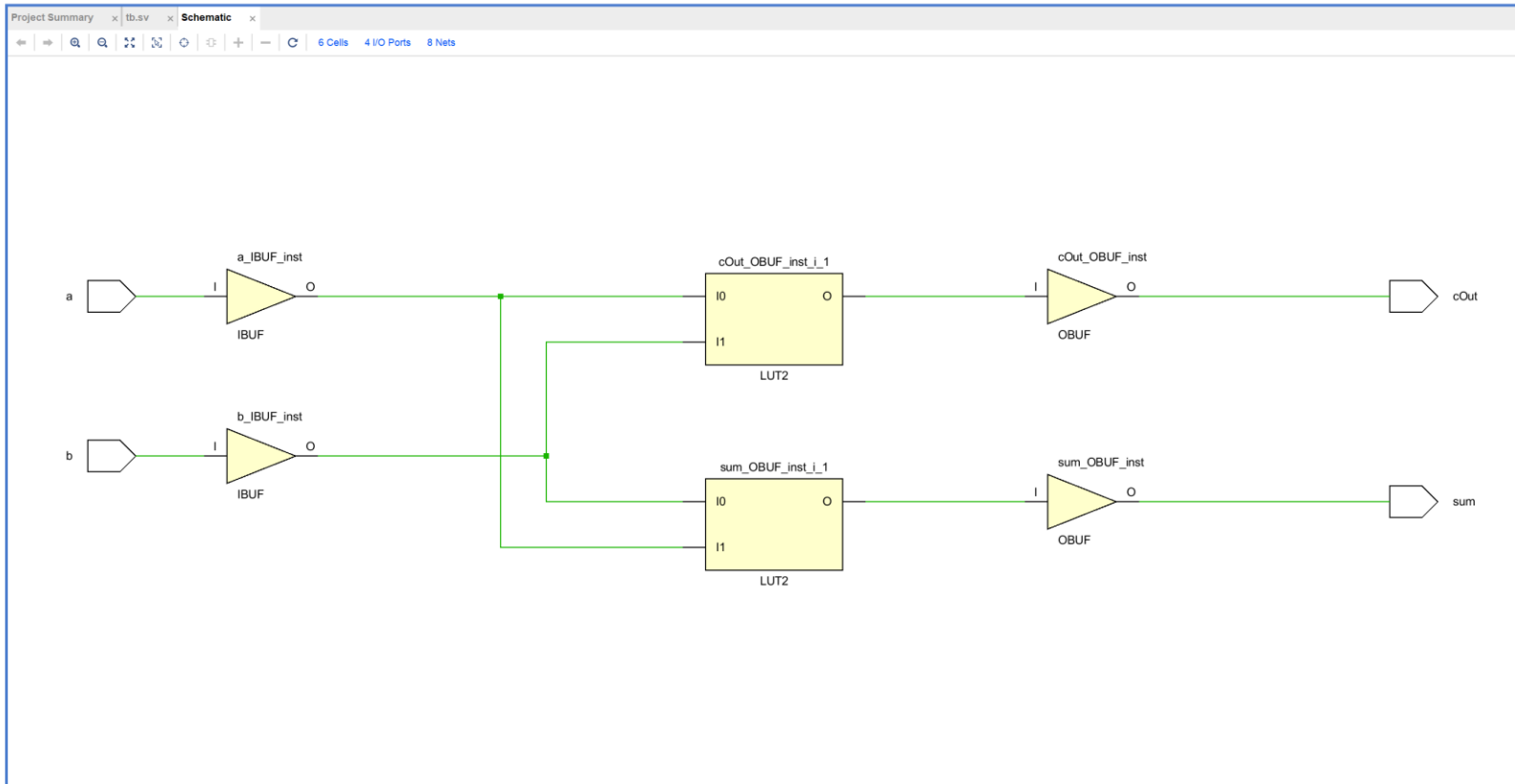
Title: ☒ CLBLL_L_X2Y1

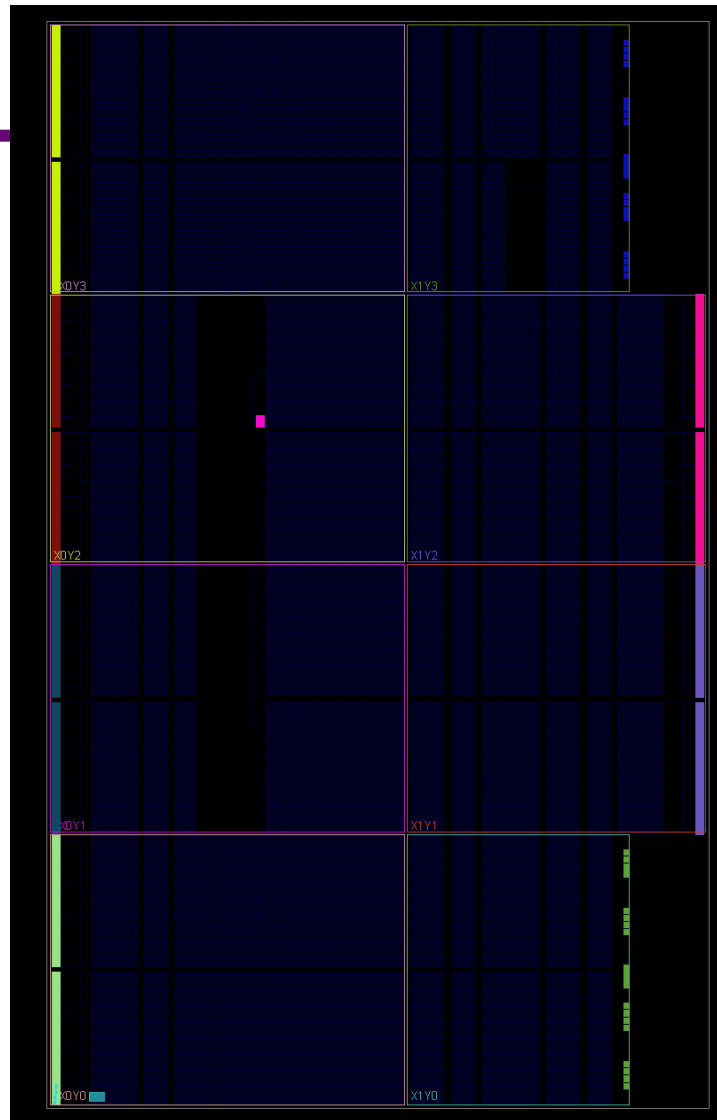
Clock region: ☒ X0Y0

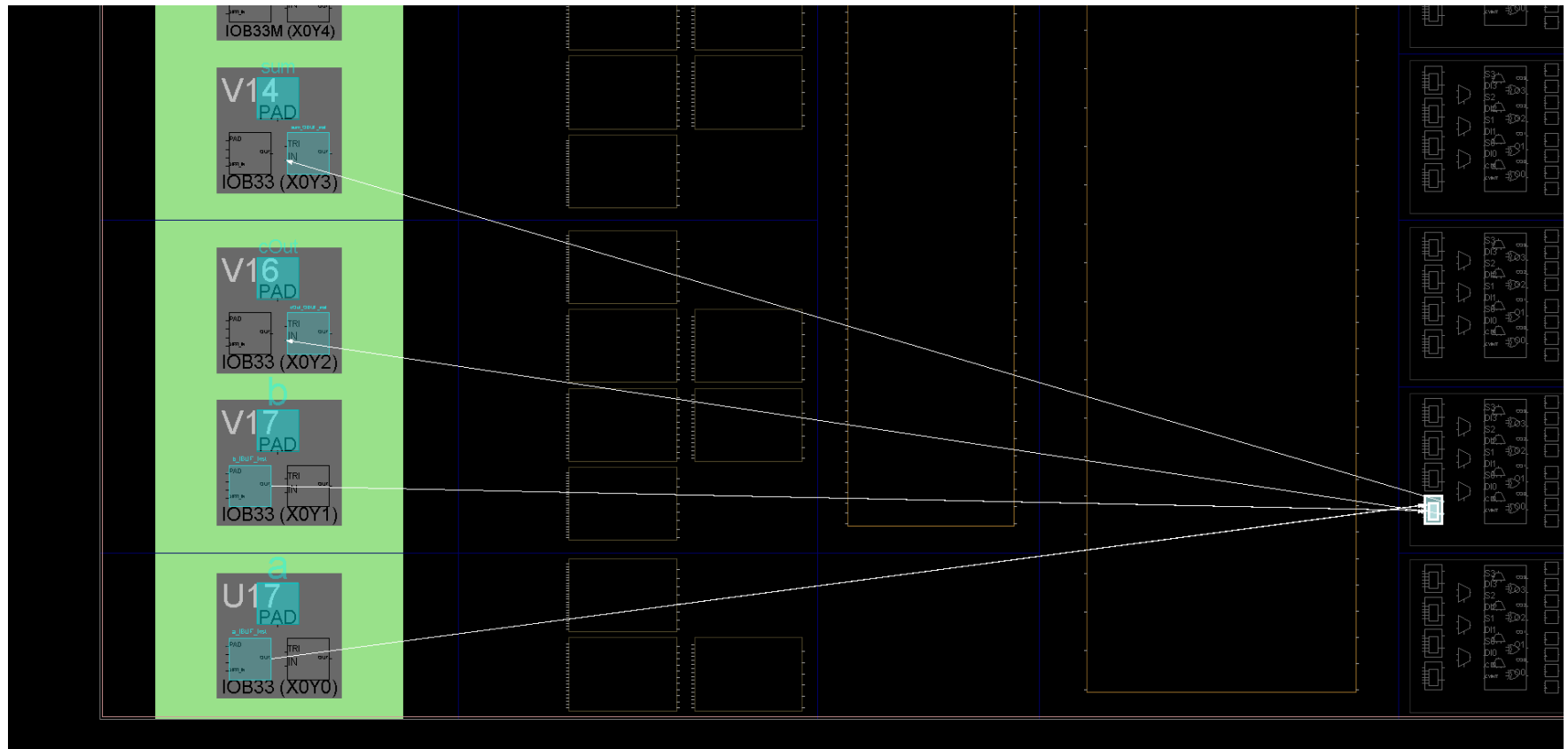
Number of cell pins: 3

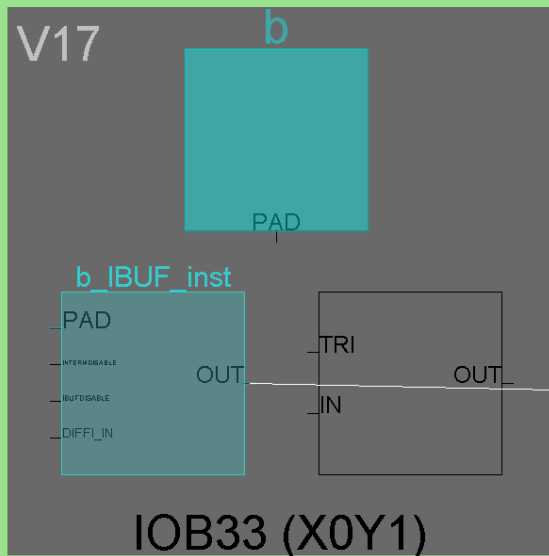
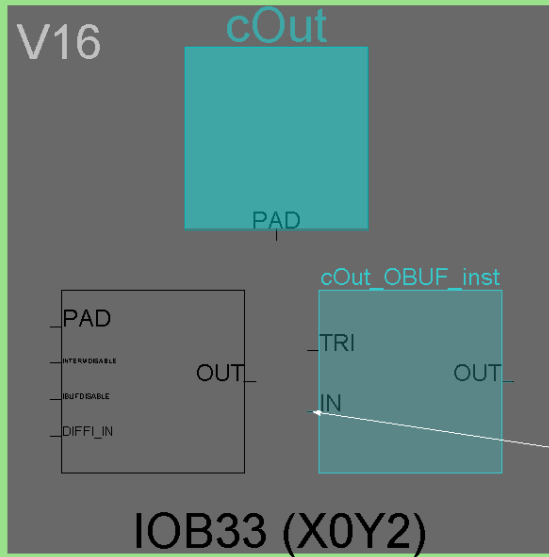
Number of nets: 3

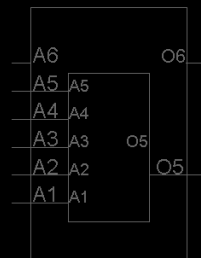
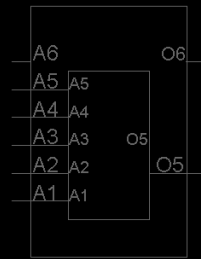
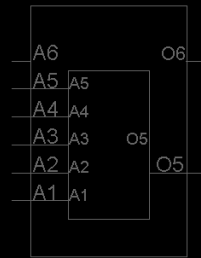
General Properties Power Nets Cell Pins



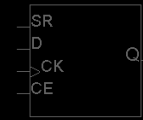
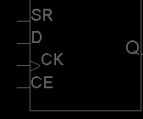
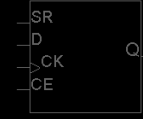
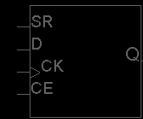
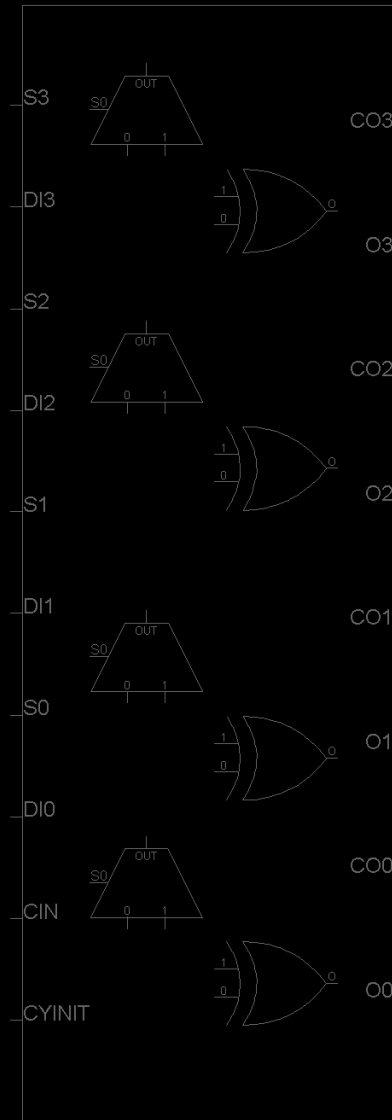
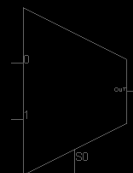
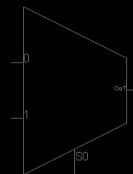
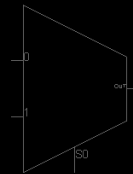
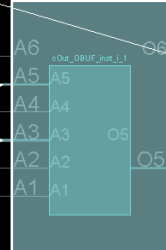








sum OBUF inst i 1



Cell Properties

Name: cOut_OBUF_inst_i_1

Reference name: LUT2

Type: LUT

BEL: ☒ A5LUT ☐ Fixed

Site: ☒ SLICE_X0Y1

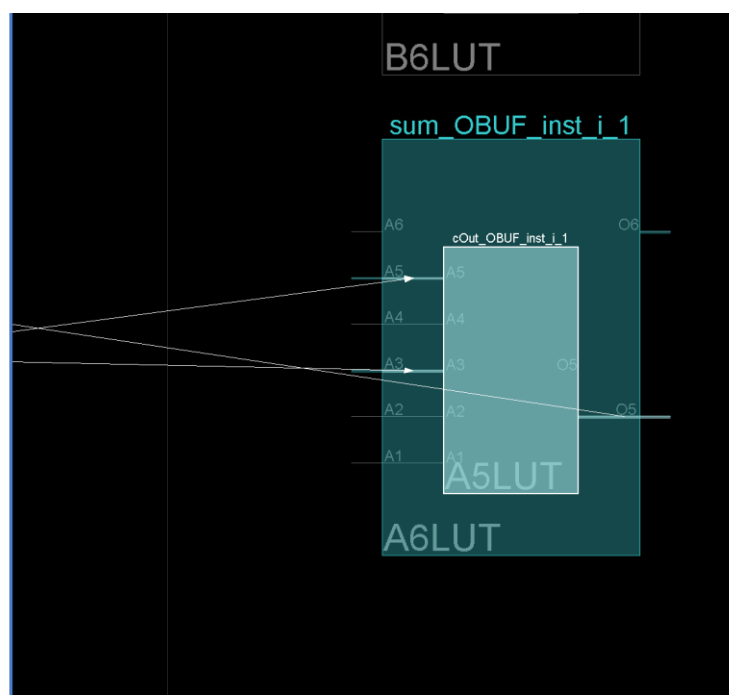
Tile: ☒ CLBLL_L_X2Y1

Clock region: ☒ X0Y0

Number of cell pins: 3

Number of nets: 3

General Properties Power Nets Cell Pins



Cell Properties

cOut_OBUF_inst_i_1

I1	I0	O=I0 & I1
0	0	0
0	1	0
1	0	0
1	1	1

Cell Properties

Name: sum_OBUF_inst_i_1

Reference name: LUT2

Type: LUT

BEL: ☒ A6LUT ☐ Fixed

Site: ☒ SLICE_X0Y1

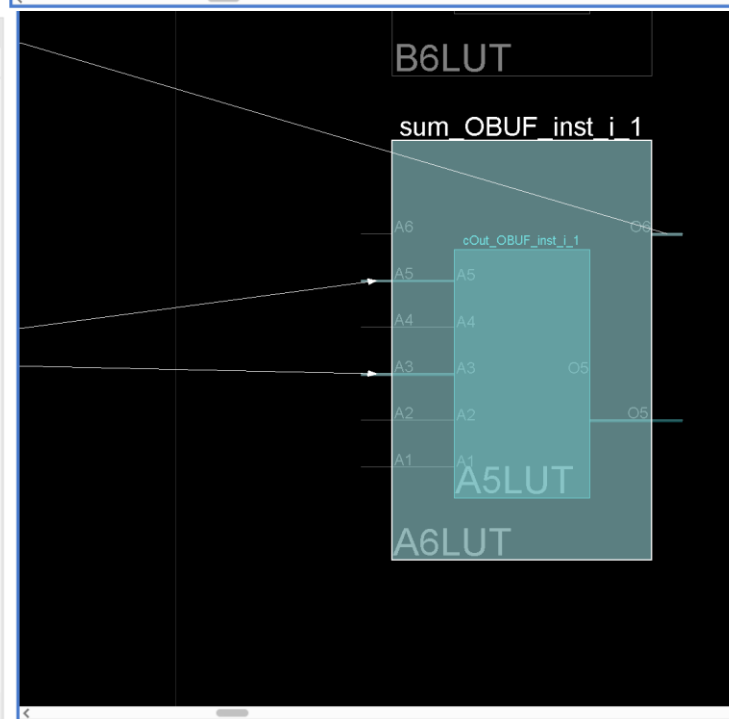
Tile: ☒ CLBLL_L_X2Y1

Clock region: ☒ X0Y0

Number of cell pins: 3

Number of nets: 3

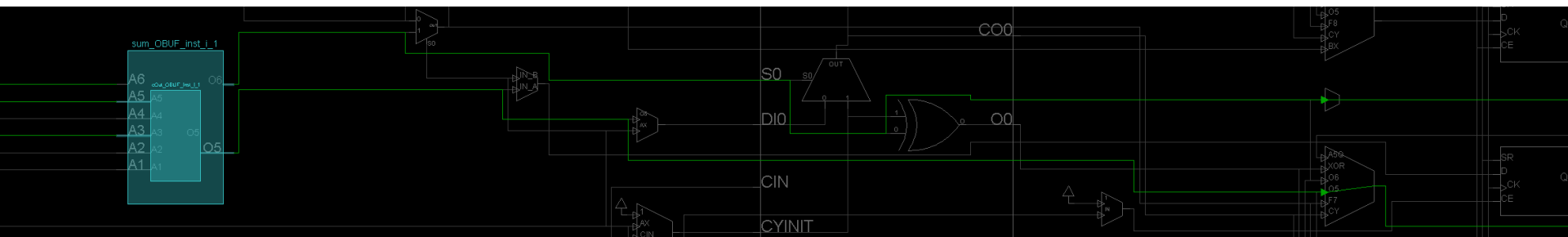
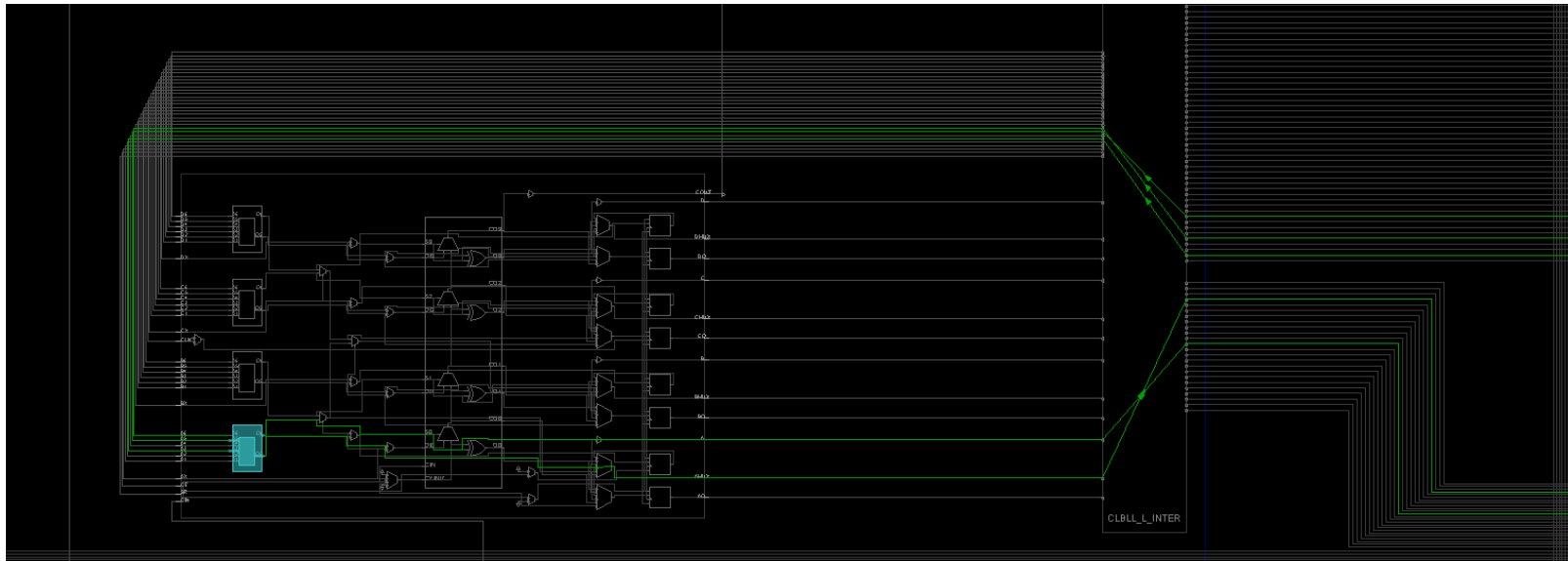
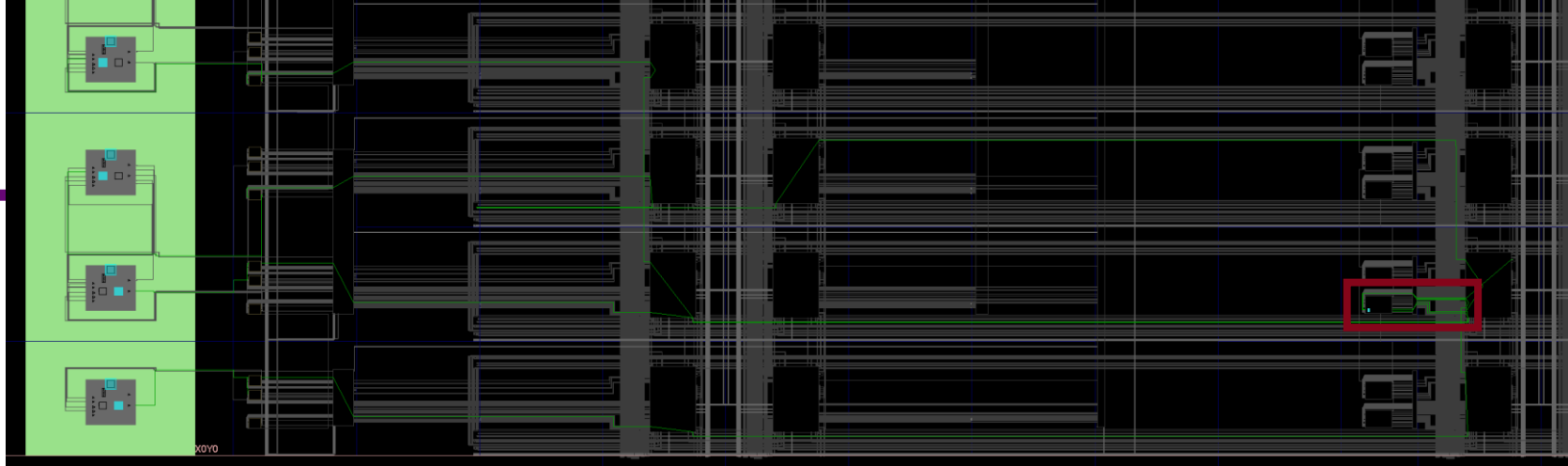
General Properties Power Nets Cell Pins



Cell Properties

sum_OBUF_inst_i_1

I1	I0	O=I0 & !I1 + !I0 & I1
0	0	0
0	1	1
1	0	1
1	1	0



▼ SIMULATION

Run Simulation

RTL ANALYSIS

> Open Elal

SYNTHESIS

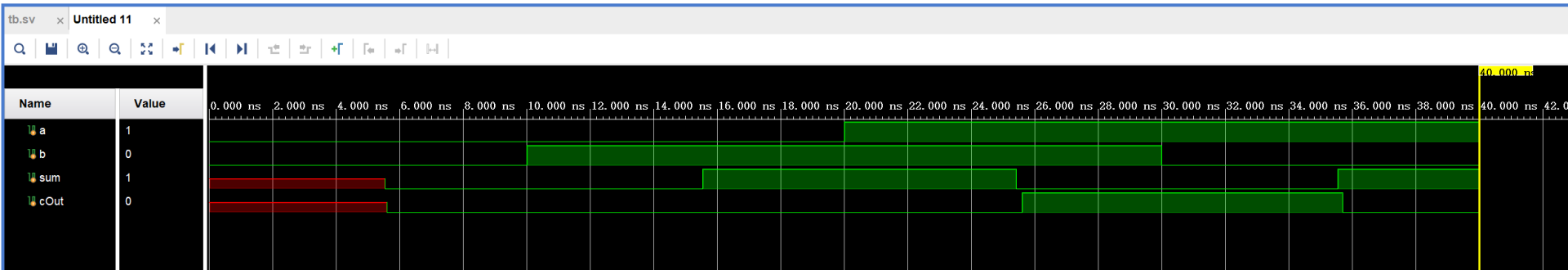
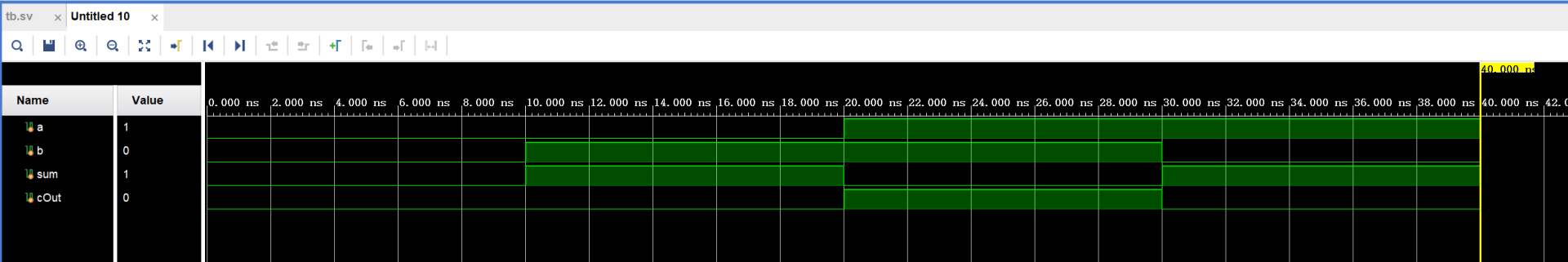
Run Behavioral Simulation

Run Post-Synthesis Functional Simulation

Run Post-Synthesis Timing Simulation

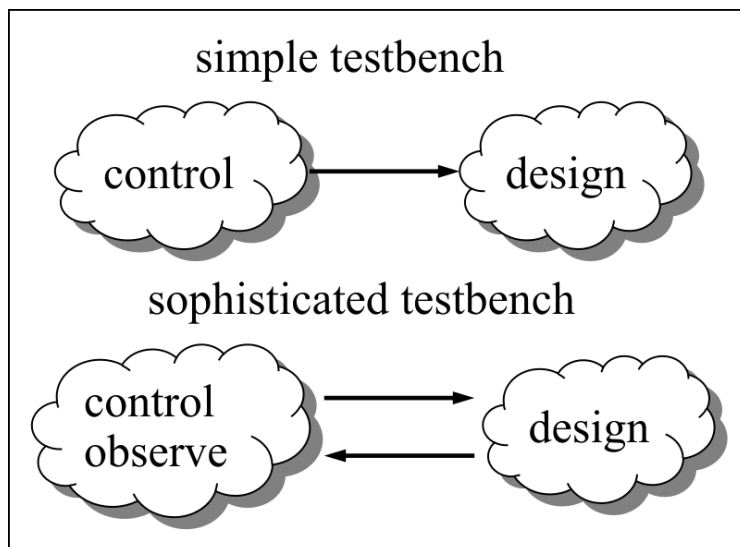
Run Post-Implementation Functional Simulation

Run Post-Implementation Timing Simulation



Testbench 功能

- ❑ 基本功能：产生待测设计的各种输入信号
- ❑ 高级功能：检查待测设计的输出信号
- ❑ 无需综合到硬件，直接由仿真器执行
 - 可以使用任何 SystemVerilog 支持的语法



SystemVerilog 仿真语言

□ 调试信息输出

■ \$display

- 用法类似 C 的 printf() 函数，行尾自带换行
- 例子：\$display("time=%0t: a = %d", \$time, a);
- 可用于仿真时输出少量信号状态信息

■ \$monitor

- 调用一次，持续监控一组信号变化
- 例子：\$monitor("time=%0t, a changed to %d", \$time, a);

■ \$time

- 系统任务，返回当前仿真时间（单位为时间精度）
- 使用 "%t" 或 "%0t" 输出

SystemVerilog 仿真语言

□ 仿真控制语句

■ initial 块

- 在仿真开始后顺序执行
- 结合延时控制，产生 DUT 的输入信号

■ always 块

- 仿真开始后永远重复执行
- 结合延时控制，生成时钟等信号

■ \$finish

- 通常用于 initial 末尾，结束仿真

```
initial begin
    clk = 0;
    clr = 0;
    x = 0;

    #5 clr = 1;
    x = 1;
    #5 clr = 0;
    #5 x = 0;
    #10 x = 0;

    #20 $finish;
end

always begin
    #5 clk = ~clk;
end
```

SystemVerilog 仿真语言

□ 延时控制

■ timescale 语句

- 用于文件头部，指定仿真的 时间单位 与 时间精度

- `timescale 1ns / 1ps: 时间单位 1ns, 时间精度 1ps

- 注意事项:

- 1) 时间单位和时间精度只能是1、10和100这三种整数，单位有s、ms、us、ns、ps和fs;

- 2) 时间精度必须小于等于时间单位

SystemVerilog 仿真语言

□ 延时控制

■ 延时语句

- #time 表示延时 time 个时间单位
- #10; // 等待 10 个时间单位，执行下一行
- #20 button = 1; // 等待 10 个单位后，执行赋值操作
- 建议只用阻塞赋值=

```

module tb;
  logic a, b, c;

  initial begin
    $monitor("t=%3d a=%b, b=%b, c=%b",
      $time, a, b, c);

    a = 1'b0;
    b = 1'b0;
    c = 1'b1;

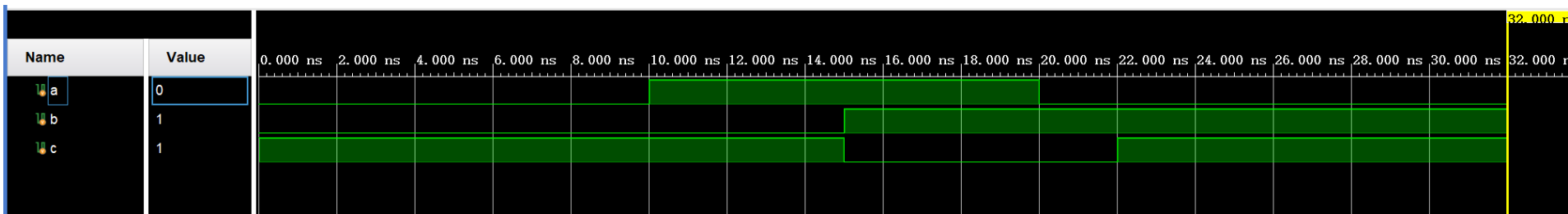
    #10
    a = 1'b1;
    b = #5 1'b1;
    c = 1'b0;

    #5
    a = 1'b0;
    c = #2 1'b1;

    #10 $finish;
  end
endmodule

```

b=#5 1b1,使其后时钟推移, 相当于#5 b=1'b1



```

module tb;
  logic a, b, c;

  initial begin
    $monitor("t=%3d a=%b, b=%b, c=%b",
      $time, a, b, c);

    a = 1'b0;
    b = 1'b0;
    c = 1'b1;

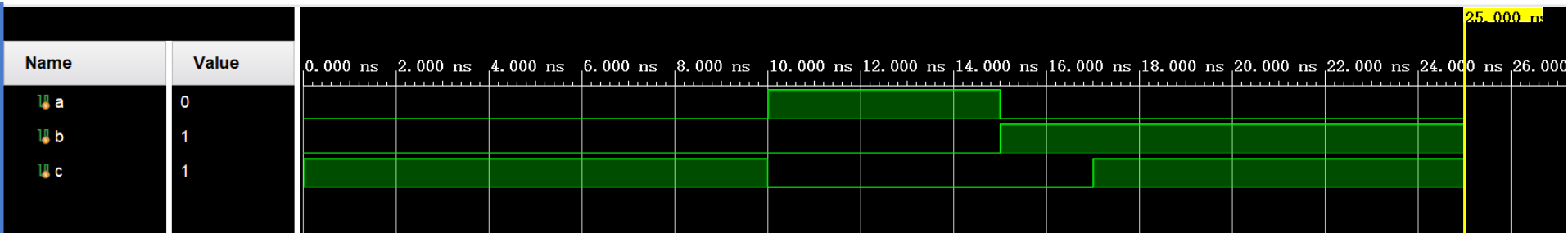
    #10
    a = 1'b1;
    b <= #5 1'b1;
    c = 1'b0;

    #5
    a <= 1'b0;
    c <= #2 1'b1;

    #10 $finish;
  end
endmodule

```

b<=#5 1'b1;只是本句延迟5单位，其后语句不受影响



对于内延时，<=对其后语句没有影响，而=使其后语句延迟若干个单位

SystemVerilog 仿真语言

□ 延时控制

■ 事件等待

- `@(posedge a);` // 等待信号 a 的上升沿，执行下一行
- `@(a) $display("a changed");` // 等待信号 a 变化，执行语句
- 也可以使用 `or`，等待多个事件


```

module tb;
  logic clk,a, b, c;

  always #1 clk = ~clk;

  initial begin
    clk = 1'b0;

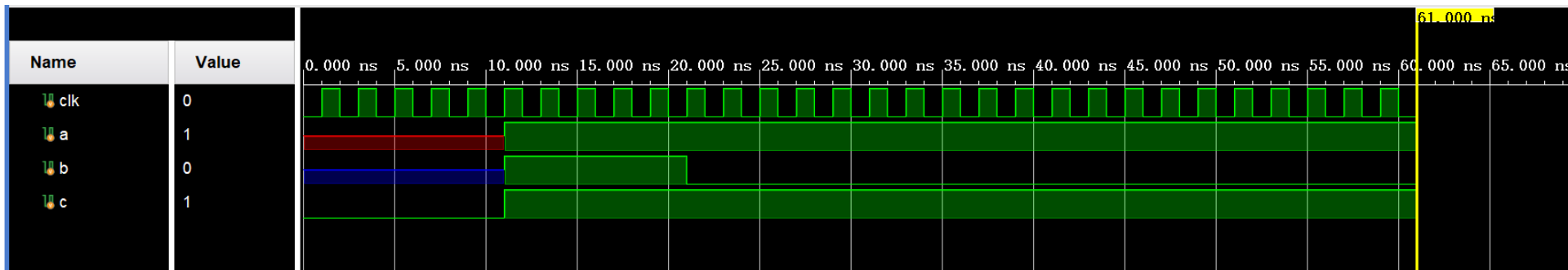
    a = 1'bX;
    b = 1'bZ;
    c = 1'b0;

    #10
    @(posedge clk);
    a = 1'b1;
    b = 1'b1;
    c = 1'b1;

    #10
    b = 1'b0;
  end

  initial begin
    @(a or b) $display("%0t, a or b changed", $time);
    #50 $finish;
  end
endmodule

```



```

# run 1000ns
11000, a or b changed
$finish called at time : 61 ns : File "G:/testprj/testprj.srscs/sources_1/new/tb.sv" Line 48
INFO: [USF-XSim-96] XSim completed. Design snapshot 'tb_behav' loaded.

```

SystemVerilog 仿真语言

□ 任务和函数

■ function 函数

- 进行简单的计算，返回单个数据
- 不能占用仿真时间，不能调用 task

```
function [7:0] sum (input [7:0] a, b);  
begin  
    sum = a + b;  
end  
endfunction
```

■ task 任务

- 完成一系列操作，返回（多个）结果
- 可以包含延时控制等
- 可以调用其他函数或任务

```
task sum;  
input [7:0] a, b;  
output [7:0] c;  
  
begin  
    @(posedge clk);  
    c = a + b;  
end  
endtask
```

```

module tb;
  logic clk,a, b, c;
  always #1 clk = ~clk;
  initial begin
    clk = 1'b0;
    a = 1'bX;
    b = 1'bZ;
    #10
    a = 1'b1;
    b = 1'b1;
    #10
    b = 1'b0;
  end

  initial begin
    #50 $finish;
  end
end

```

```

always @(posedge clk) sum(a,b,c);

```

```

task sum;
  input x, y;
  output z;

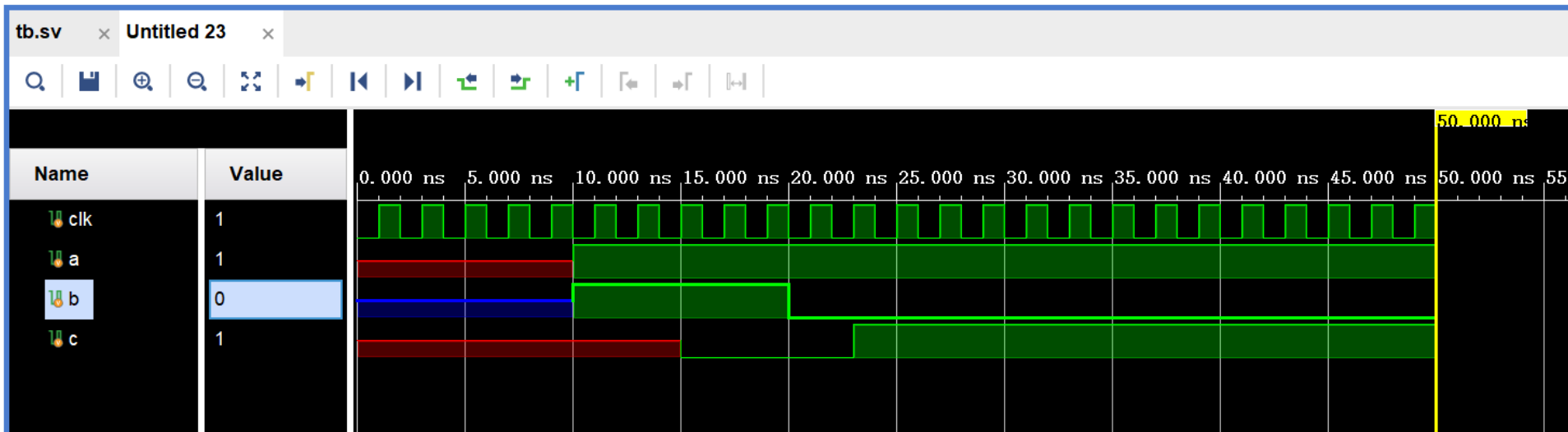
  begin
    @(posedge clk);
    z = x + y;
  end
endtask

```

```

endmodule

```



SystemVerilog 仿真语言

□ 控制流语句

- while, for 循环
- repeat(x) 循环 x 次
- fork-join 语句（不常用）

□ 更多数据结构

- 动态数组、关联数组

```

module tb;
  logic clk,a, b, c;
  always #1 clk = ~clk;

```

```

initial begin
  clk = 1'b0;
  a = 1'bX;
  b = 1'bZ;

  #10;
  @(posedge clk);
  a = 1'b1;
  b = 1'b1;

  repeat(2) @(posedge clk);
  b = 1'b0;

  #10
  b = 1'b1;
end

```

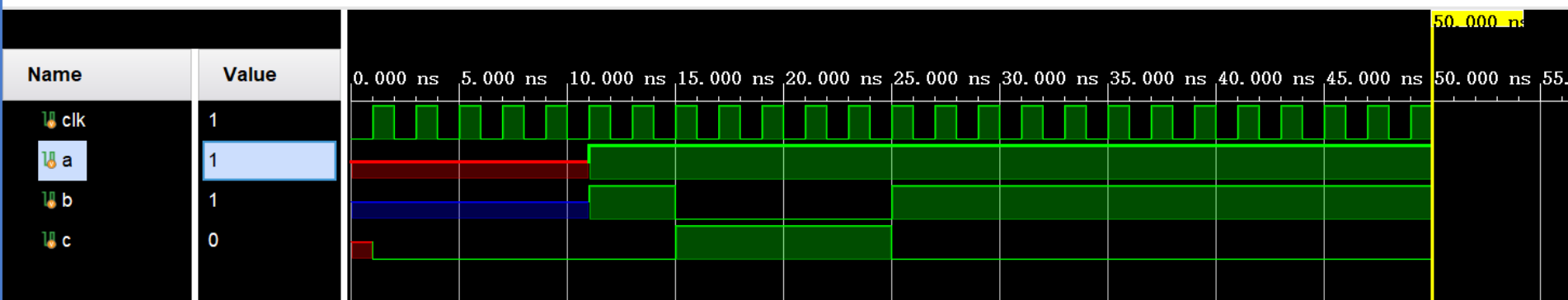
```

always@(posedge clk) begin
  if(b == 1'b0)
    c = 1'b1;
  else
    c = 1'b0;
end

initial begin
  #50 $finish;
end

endmodule

```



SystemVerilog 仿真语言

□ 断言机制

- `assert(<expression>);` // `expression = 0` 时报错
- 便利地验证输出与预期的一致性
- 提供更高級的断言机制，可自学 SVA 语言

```

module tb;
  logic clk,a, b, c;
  always #1 clk = ~clk;

```

```

initial begin

```

```
  clk = 1'b0;
```

```
  a = 1'bX;
```

```
  b = 1'b0;
```

```
  #10;
```

```
  @(posedge clk);
```

```
  a = 1'b1;
```

```
  b = 1'b1;
```

```
  repeat(2) @(posedge clk);
```

```
  b = 1'b0;
```

```
  #10
```

```
  b = 1'bZ;
```

```
  #1
```

```
  b = 1'b1;
```

```
end
```

```

always@(posedge clk) begin

```

```
  assert(b !== 1'bz);
```

```
  if(b == 1'b0)
```

```
    c = 1'b1;
```

```
  else
```

```
    c = 1'b0;
```

```
end
```

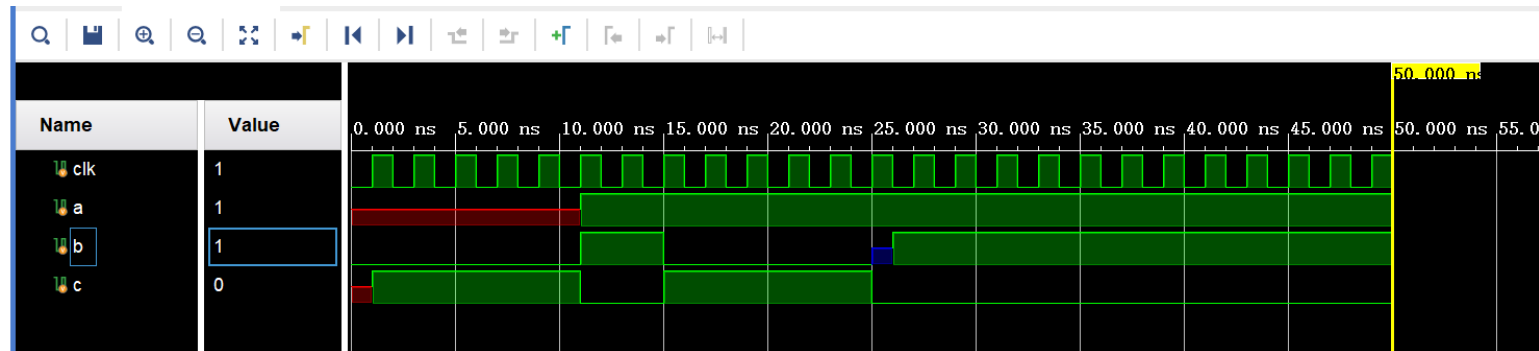
```

initial begin

```

```
  #50 $finish;
```

```
end
```



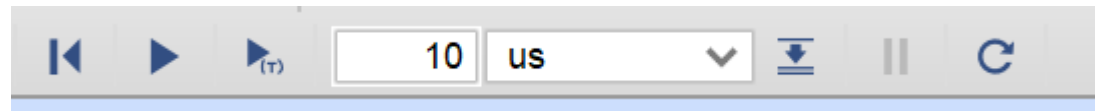
Time resolution is 1 ps

Error: Assertion violation

Time: 25 ns Iteration: 0 Process: /tb/Always50_2 File: G:/testprj/testprj.srscs/sources_1/new/tb.sv

vivado仿真调试

- 控制台输出
- 查看波形
- 设置断点
- 断言




```

23 module tb;
24     logic clk,a, b, c;
25
26 ○ always #1 clk = ~clk;
27
28     initial begin
29 ○     clk = 1'b0;
30
31 ○     a = 1'bX;
32 ○     b = 1'b0;
33
34 ○     #10;
35 ○     @(posedge clk);
36 ○     a = 1'b1;
37 ○     b = 1'b1;
38
39 ○     repeat(2) @(posedge clk);
40 ○     b = 1'b0;
41
42 ○     #10
43 ○     b = 1'bZ;
44
45 ○     #1
46 ○     b = 1'b1;
47
48     end
49
50 ○ always@(posedge clk) begin
51 ○     assert(b !== 1'bZ);
52 ○     if(b == 1'b0)
53 ○         c = 1'b1;
54 ○     else
55 ○         c = 1'b0;
56     end
57
58     initial begin
59 ○ → #50 $finish;
60     end
61
62 endmodule
63

```

```

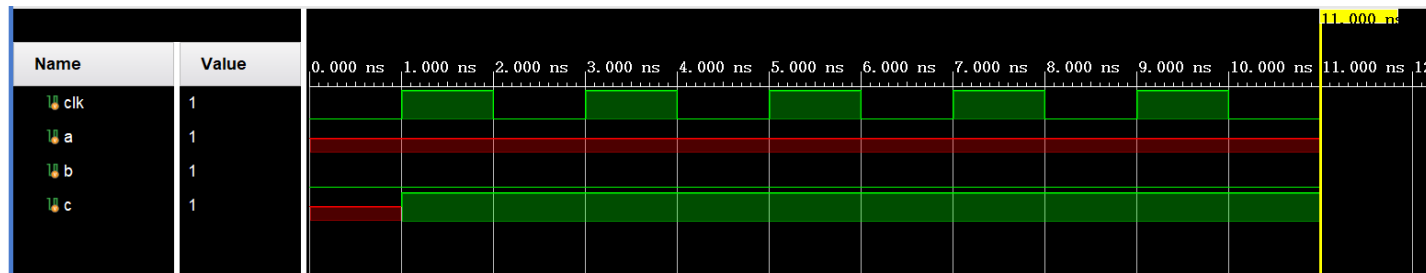
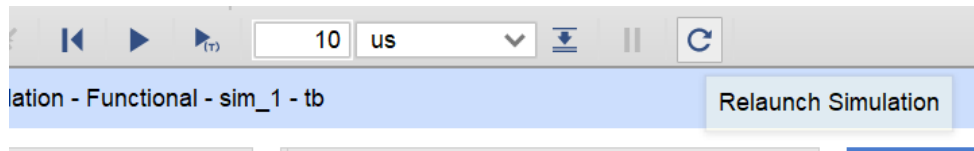
23 module tb;
24     logic clk,a, b, c;
25
26 ○ always #1 clk = ~clk;
27
28     initial begin
29 ○     clk = 1'b0;
30
31 ○     a = 1'bX;
32 ○     b = 1'b0;
33
34 ○     #10;
35 ○     @(posedge clk);
36 ○     a = 1'b1;
37 ○     b = 1'b1;
38
39 ○     repeat(2) @(posedge clk);
40 ●     b = 1'b0;
41
42 ○     #10
43 ○     b = 1'bZ;
44
45 ○     #1
46 ○     b = 1'b1;
47
48     end
49
50 ○ always@(posedge clk) begin
51 ○     assert(b !== 1'bZ);
52 ○     if(b == 1'b0)
53 ○         c = 1'b1;
54 ○     else
55 ●     c = 1'b0;
56     end
57
58     initial begin
59 ○ → #50 $finish;
60     end
61
62 endmodule
63

```

```

22
23 module tb;
24     logic clk, a, b, c;
25
26     always #1 clk = ~clk;
27
28     initial begin
29         clk = 1'b0;
30
31         a = 1'bX;
32         b = 1'b0;
33
34         #10;
35         @(posedge clk);
36         a = 1'b1;
37         b = 1'b1;
38
39         repeat(2) @(posedge clk);
40         b = 1'b0;
41
42         #10
43         b = 1'bZ;
44
45         #1
46         b = 1'b1;
47
48     end
49
50     always@(posedge clk) begin
51         assert(b != 1'bZ);
52         if(b == 1'b0)
53             c = 1'b1;
54         else
55             c = 1'b0;
56     end
57
58     initial begin
59         #50 $finish;
60     end
61
62 endmodule
63

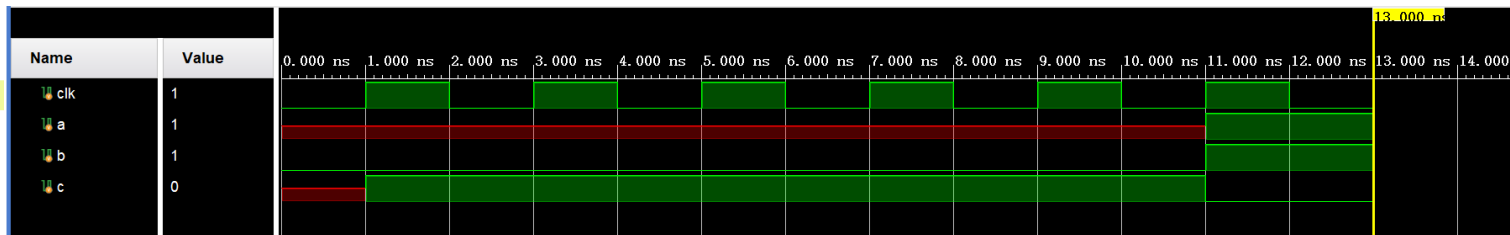
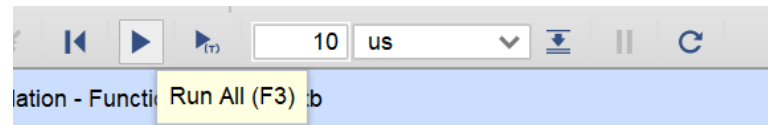
```



```

23 module tb;
24     logic clk, a, b, c;
25
26     ○ always #1 clk = ~clk;
27
28     initial begin
29         ○ clk = 1'b0;
30
31         ○ a = 1'bX;
32         ○ b = 1'b0;
33
34         ○ #10;
35         ○ @(posedge clk);
36         ○ a = 1'b1;
37         ○ b = 1'b1;
38
39         ○ repeat(2) @(posedge clk);
40         ● b = 1'b0;
41
42         ○ #10
43         ○ b = 1'bZ;
44
45         ○ #1
46         ○ b = 1'b1;
47
48     end
49
50     always@(posedge clk) begin
51         ○ assert(b !== 1'bZ);
52         ○ if(b == 1'b0)
53         ○     c = 1'b1;
54         ○ else
55         ● → c = 1'b0;
56     end
57
58     initial begin
59         ○ #50 $finish;
60     end
61
62 endmodule

```



实验模板中的仿真环境

□ Testbench 模板文件：sim_1/new/tb.sv

- 例化 DUT：thinpad_top 模块
- Testbench 模块相当于虚拟的实验板
 - 自定义按钮、拨码开关的输入操作
 - 初始化 SRAM、Flash 等内存资源
 - 模拟云平台（或 PC）对串口的发送操作
- 其他辅助模块
 - clock.v 产生 50MHz、11.0592MHz 时钟信号
 - sram_model.v SRAM 芯片的仿真模型
 - cpld_model.v CPLD 串口的仿真模型（本学期不用）
 - 28F640P30.v Flash 芯片的仿真模型

实验模板中的仿真环境

```
// Windows需要注意路径分隔符的转义, 例如"D:\\foo\\bar.bin"
parameter BASE_RAM_INIT_FILE = "/tmp/main.bin"; // BaseRAM初始化文件, 请修改为实际的绝对路径
parameter EXT_RAM_INIT_FILE = "/tmp/eram.bin"; // ExtRAM初始化文件, 请修改为实际的绝对路径
parameter FLASH_INIT_FILE = "/tmp/kernel.elf"; // Flash初始化文件, 请修改为实际的绝对路径
```

```
assign rxd = 1'b1; // idle state
initial begin
    // 在这里可以自定义测试输入序列, 例如:
    dip_sw = 32'h2;
    touch_btn = 0;
    reset_btn = 1;
    #100;
    reset_btn = 0;
    for (integer i = 0; i < 20; i = i+1) begin
        #100; // 等待100ns
        push_btn = 1; // 按下手工时钟按钮
        #100; // 等待100ns
        push_btn = 0; // 松开手工时钟按钮
    end
    // 模拟PC通过串口发送字符
    cpld.pc_send_byte(8'h32);
    #10000;
    cpld.pc_send_byte(8'h33);
end
```

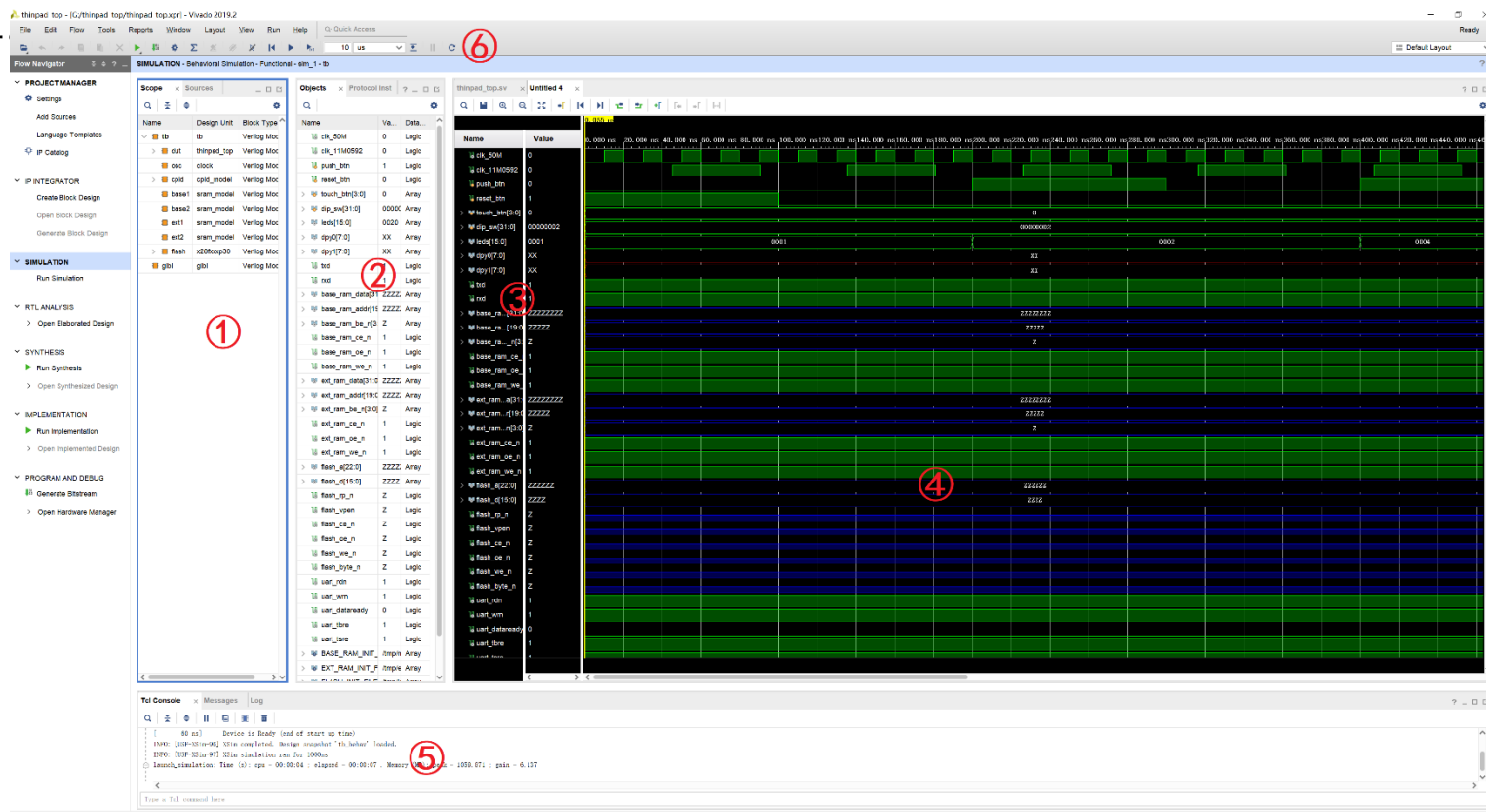
实验模板中的仿真环境

```
74 // 待测试用户设计
75 > thinpad_top dut( ...
112 );
113 // 时钟源
114 > clock osc( ...
117 );
118 // CPLD 串口仿真模型
119 > cpld_model cpld( ...
127 );
128 // BaseRAM 仿真模型
129 > sram_model base1(/*autoinst*/ ...
137 > sram_model base2(/*autoinst*/ ...
145 // ExtRAM 仿真模型
146 > sram_model ext1(/*autoinst*/ ...
154 > sram_model ext2(/*autoinst*/ ...
162 // Flash 仿真模型
163 > x28fxxp30 #(.FILENAME_MEM(FLASH_INIT_FILE)) flash( ...
178 > initial begin ...
183 end
184
185 // 从文件加载 BaseRAM
186 > initial begin ...
205 end
206
207 // 从文件加载 ExtRAM
208 > initial begin ...
227 end
228 endmodule
229
```

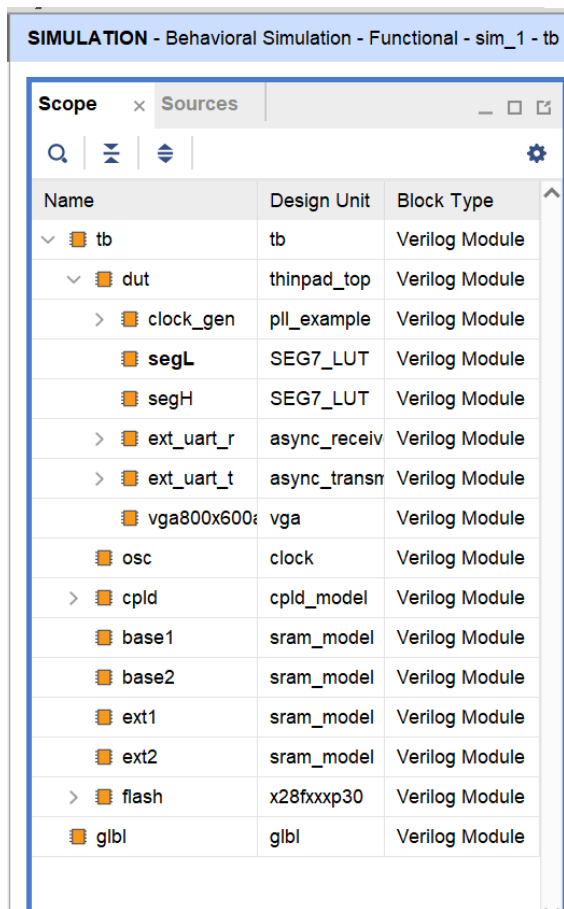
`dut`就是 `thinpad_top` 的一个实例化。其它的是外围的仿真模型，包括串口，静态内存，FLASH 等。在顶层项目中，这些内容都已经被连接完成了，直接使用即可。

仿真界面

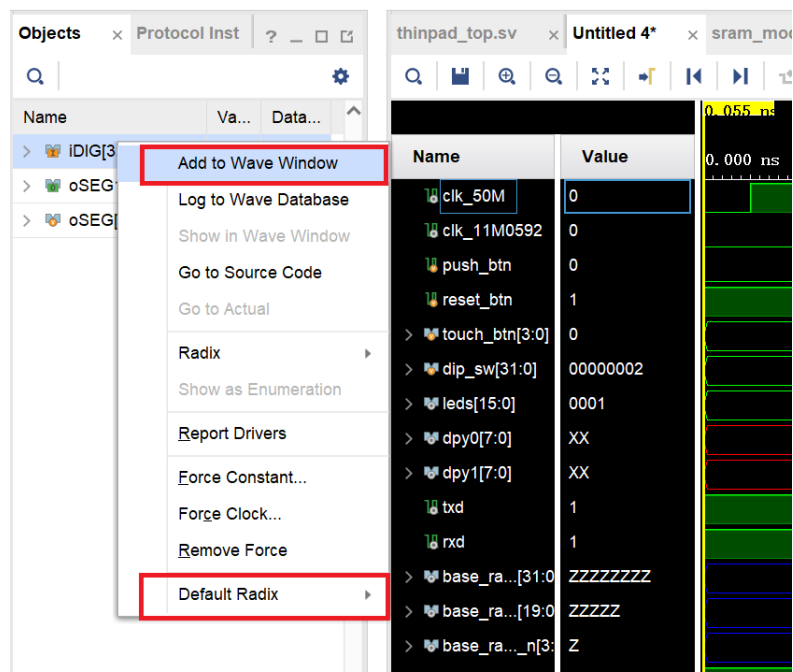
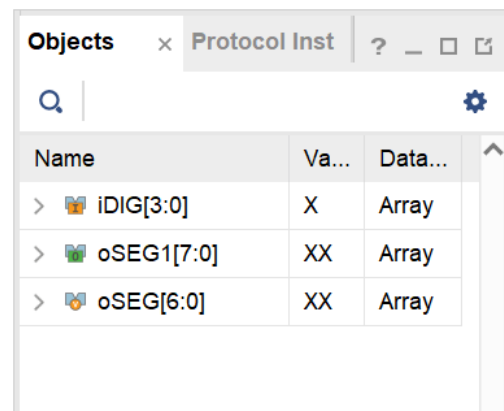
- ① 选择对象的范围；
- ② 某一个具体对象的信号；
- ③ 波形图的信号；
- ④ 波形图；
- ⑤ 命令行窗口（这里是可以敲入命令的，大部分情况下不需要，但是有的时候是必须的）；
- ⑥ 仿真的命令菜单

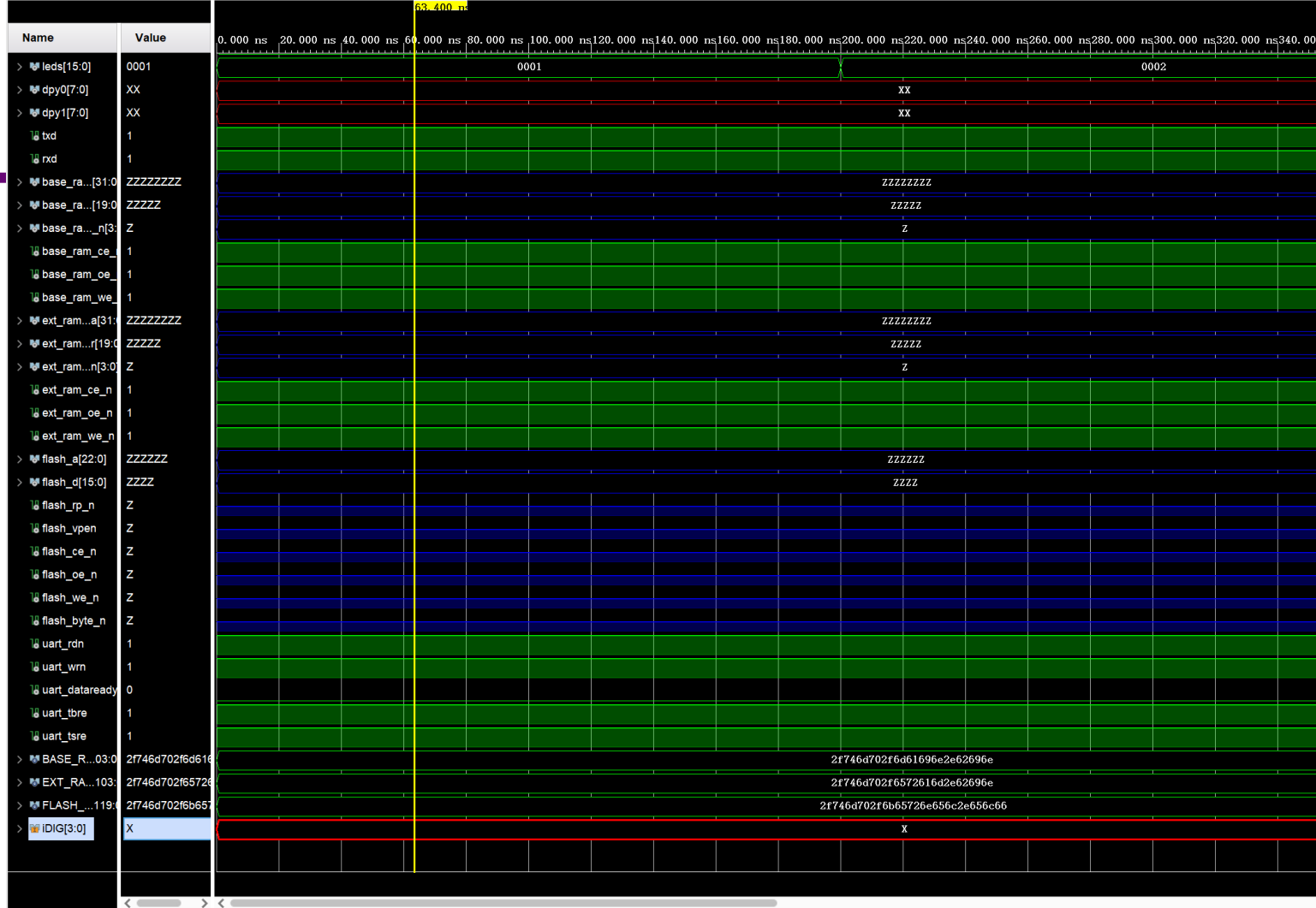


①选择需要观察的对象，对象的名称与源文件中一致的。在②中就显示了对应的对象的信号，包括了内部的信号以及对外的输入输出的信号。



如果希望把某一个信号加入到波形列表中，可以右键点击对应的信号，选择加入观察波形中。





左边是波形图中显示的信号，右面就是对应信号的波形图，最上面给出了显示波形的时间序列关系。右键点击波形图信号，可以查看对应信号的操作，包括可以删除改名等。右边的波形图就显示了各个信号随着时间变化的过程，蓝色的表示 Z，即高阻态，红色的代表 X，即未定义值，一般也就代表了模块中未初始化的部分，或者代表了代码中有错误。应该要特别注意按下 **reset** 按钮之后出现的红色信号的部分，往往这意味着代码的错误。

一些提示

□ 区分硬件描述与仿真语言

- 可综合的逻辑是 SystemVerilog 子集
- 前述的大多数语句只能用于 Testbench

□ 灵活封装常用操作

- 使用 function 或宏定义实现简单计算
 - 本实验中的操作指令生成
- 使用 task 定义可复用的时间序列操作
 - 多个按钮操作
 - 串口发送一系列数据（结合数组）
 - Wishbone 总线发送数据

□ 使用断言机制

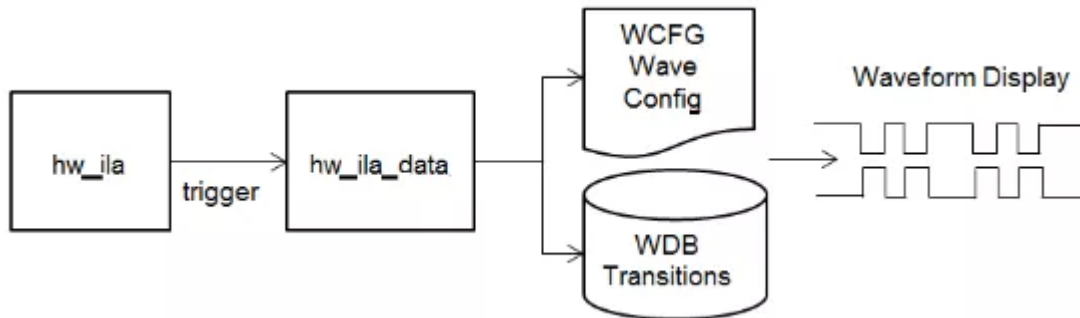
- 自动判断错误，减少阅读波形的时间

时序仿真

- 有时候行为仿真不能够查出所有问题
- 可能需要时序仿真
- 时序仿真要比行为（逻辑）仿真慢的多，消耗更多资源
- 本课程实验通常情况下行为仿真已经足够

上板调试

- 外部信号
- 内部信号
- 逻辑分析仪（LA）
- ILA（Integrated Logic Analyzer）



```

module clock (
    input wire clock,
    input wire rst,
    output reg[6:0] seg1,
    output reg[6:0] seg2
);

    reg[3:0] cnt_L = 4'b0000;
    reg[3:0] cnt_H = 4'b0000;
    reg[25:0] cnt = 26'b000000000000000000000000;
    reg clk_out;
    reg[3:0] tmp_H = 4'b0000;
    reg[3:0] tmp_L = 4'b0000;

    always_ff @ (posedge clock) begin
        cnt <= cnt + 1;
        if(cnt == 26'b00_0000_0000_0000_0000_0000)
            clk_out <= 0;
        if(cnt == 26'b10_1111_1010_1111_0000_1000_0000) // 计数 50000000 次, 1 秒钟
            begin
                cnt <= 26'b00_0000_0000_0000_0000_0000;
                clk_out <= 1;
            end
        end

    always_ff @ (posedge clk_out or posedge rst) begin // 将秒数转换成输出的格式
        if(rst == 1)
            begin
                tmp_H = 4'b0000;
                tmp_L = 4'b0000;
            end
        else
            begin
                tmp_L = cnt_L + 1;
                if(tmp_L > 4'b1001)
                    begin
                        tmp_L = 4'b0000;
                        tmp_H = cnt_H + 1;
                        if (tmp_H > 4'b1001)
                            tmp_H = 4'b0000;
                    end
                end
                cnt_L = tmp_L;
                cnt_H = tmp_H;
            end
        end
    end
end

```

ThinPAD-Cloud
在线实验
自动评测
lishanshan
登出

工作区域

轻轻几点, 就能完成原来实体硬件实验板上的复杂操作

串口输入/输出

清空

发送 请输入数据, 回车发送

换行符 NONE

回车发送数据时添加的换行符

接口 tx/rxd (直连串口信号)

波特率 9600

校验位 NONE

数据位 8

停止位 1

打开串口

更新设计文件

选择 CI 结果

选择文件 未选择任何文件

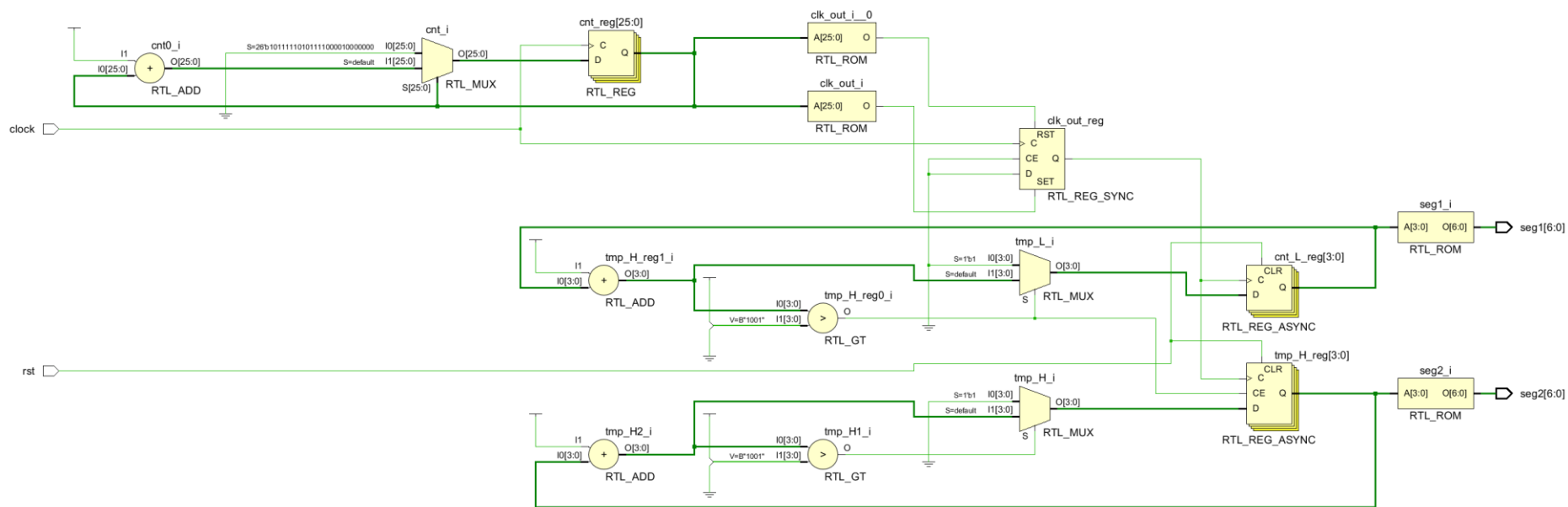
选择 .bit 文件

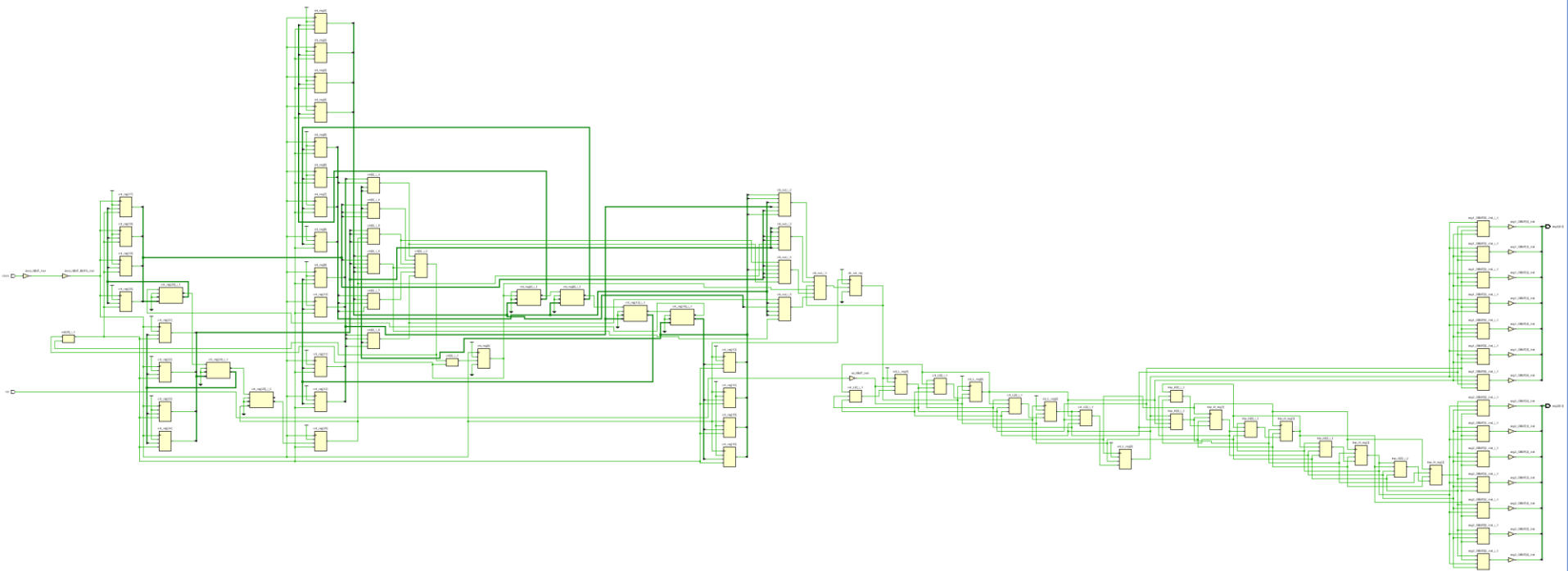
调试工具

内存总线分析 远程 JTAG 连接 内部信号采样

连接方法

本功能允许 Vivado 远程连接到 FPGA 的 JTAG, 以便支持 ILA 和 VIO 等功能。
Hardware Server IP 166.111.226.111, Port 4564
连接方法说明





ILA使用

□ 在设计中加入ILA

□ IP核→连线→采样→查看

The screenshot displays the Xilinx Vivado IDE interface. On the left, the 'Flow Navigator' pane shows the 'PROJECT MANAGER' section with 'IP Catalog' highlighted. The main workspace is titled 'IMPLEMENTED DESIGN - xc7a100tfgg676-2L'. It features a 'Sources' pane on the left showing 'Design Sources (1)' with 'clock (clock.sv)'. Below this is the 'Hierarchy' pane showing 'IP Properties' for the 'ILA (Integrated Logic Analyzer)' core, with version '6.2 (Rev. 10)' and interfaces 'AXI4, AXI4-Stream'. On the right, the 'Project Summary' pane shows the 'Cores' tab with a search for 'ILA' yielding 4 matches. The table below lists the cores:

Name	AXI4	Status	License	VLN
Vivado Repository				
Alliance Partners				
Xylon				
Multilayer Video Controller	AXI4	Production	Purchase	logicbricks.com:logicbricks:logicvc:0.0
Debug & Verification				
Debug				
ILA (Integrated Logic Analyzer)	AXI4, AXI4-Stream	Production	Included	xilinx.com:ip:ila:6.2
System ILA	AXI4, AXI4-Stream	Production	Included	xilinx.com:ip:system_ila:1.1
Video & Image Processing				
Multilayer Video Controller	AXI4	Production	Purchase	logicbricks.com:logicbricks:logicvc:0.0

□ 设置ILA

Customize IP

ILA (Integrated Logic Analyzer) (6.2)

[Documentation](#) [IP Location](#) [Switch to Defaults](#)

☐ Show disabled ports

Component Name

To configure more than 64 probe ports use Vivado Tcl Console

General Options

Monitor Type

☒ Native ☐ AXI

Number of Probes [1...1024]

Sample Data Depth

☒ Same Number of Comparators for All Probe Ports

Number of Comparators

☐ Trigger Out Port

☐ Trigger In Port

Input Pipe Stages

Trigger And Storage Settings

☐ Capture Control

☐ Advanced Trigger

GUI configuration mode is limited to 64 probe ports.

OK Cancel

clk
probe0[0:0]
probe1[0:0]

□ 设置ILA

Customize IP

ILA (Integrated Logic Analyzer) (6.2)

Documentation IP Location Switch to Defaults

☐ Show disabled ports

Component Name

To configure more than 64 probe ports use Vivado Tcl Console

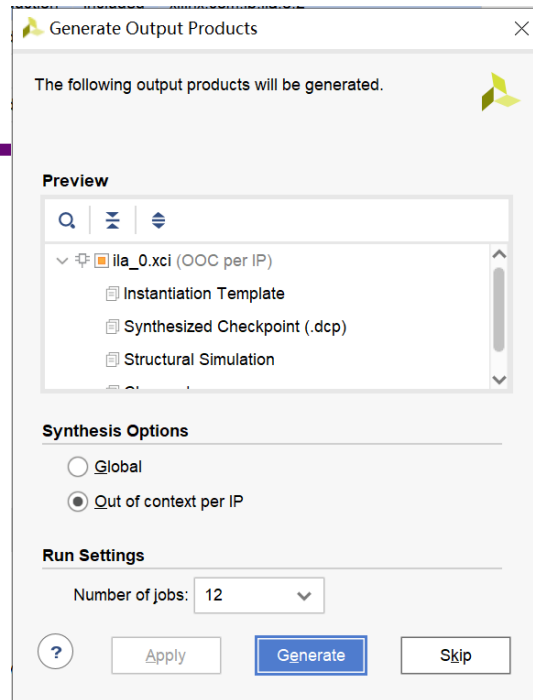
General Options Probe_Ports(0..1)

Probe Port	Probe Width [1..4096]	Number of Comparators	Probe Trigger or Data
PROBE0	10	1	DATA AND TRIGGER
PROBE1	4	1	DATA AND TRIGGER

clk
probe0[9:0]
probe1[3:0]

OK Cancel

生成ILA核



IMPLEMENTED DESIGN - xc7a100tfgg676-2L

Sources x Netlist ? _ □ □

ila_0 (ila_0.xci) (1)

- ila_0 (ila_0.v) (1)
- clock (clock.sv)

Constraints (1)

Simulation Sources (2)

Utility Sources

Source File Properties ? _ □ □ x

ila_0.xci

☒ Enabled

Location: g:/clock_ila/projects/clock/clock.srscs/sources_1/

Project Summary x Device x clock.sv x Schematic x Schematic (2) x **IP Catalog** x ila_0.v x

Cores | Interfaces

Search: Q: ILA (4 matches)

Name	AXI4	Status	License	VLNV
Vivado Repository				
Alliance Partners				
Xylon				
Multilayer Video Controller	AXI4	Production	Purchase	logicbricks.com:logicbricks:logicvcv:0.0
Debug & Verification				
Debug				
ILA (Integrated Logic Analyzer)	AXI4, AXI4-Stream	Production	Included	xilinx.com:ip:ila:6.2
System ILA	AXI4, AXI4-Stream	Production	Included	xilinx.com:ip:system_ila:1.1
Video & Image Processing				
Multilayer Video Controller	AXI4	Production	Purchase	logicbricks.com:logicbricks:logicvcv:0.0

Sources x **Netlist** ? _ □ □

🔍 ⚙️ ⚙️ ⚙️ ⚙️ 0 ⚙️

- Design Sources (2)
 - ila_0 (ila_0.xci) (1)
 - ila_0 (ila_0.v) (1)
 - inst : ila_v6_2_10_ila
 - clock (clock.sv)
- Constraints (1)
- Simulation Sources (2)
- Utility Sources

Hierarchy IP Sources Libraries Compile Order

Source File Properties

ila_0.xci

←

→

⚙

☒ Enabled

Location:

g:/clock_ila/projects/clock/clock.srcs/sources_1/

Type:

IP

...

Part:

xc7a100tfgg676-2L

Size:

401.3 KB

Modified:

Today at 23:27:35 PM

Copied to:

g:/clock_ila/projects/clock/clock.srcs/sources_1/

Read-only:

No

Encrypted:

No

Core Container:

No

Used In

☒ Synthesis

☒ Implementation

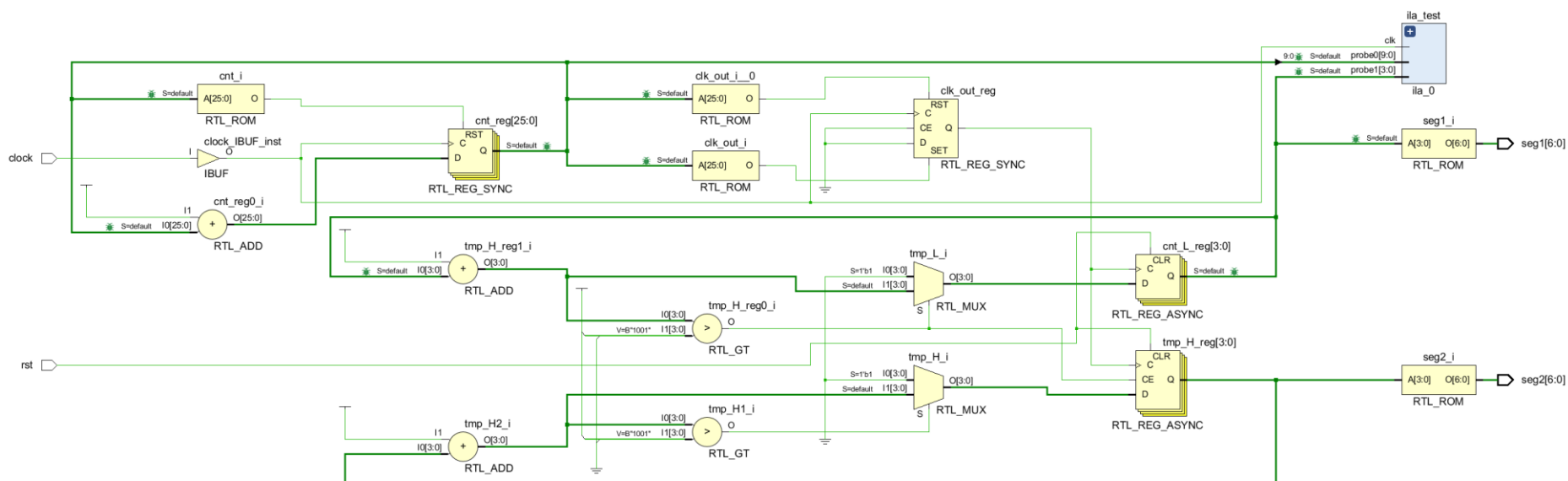
☒ Simulation

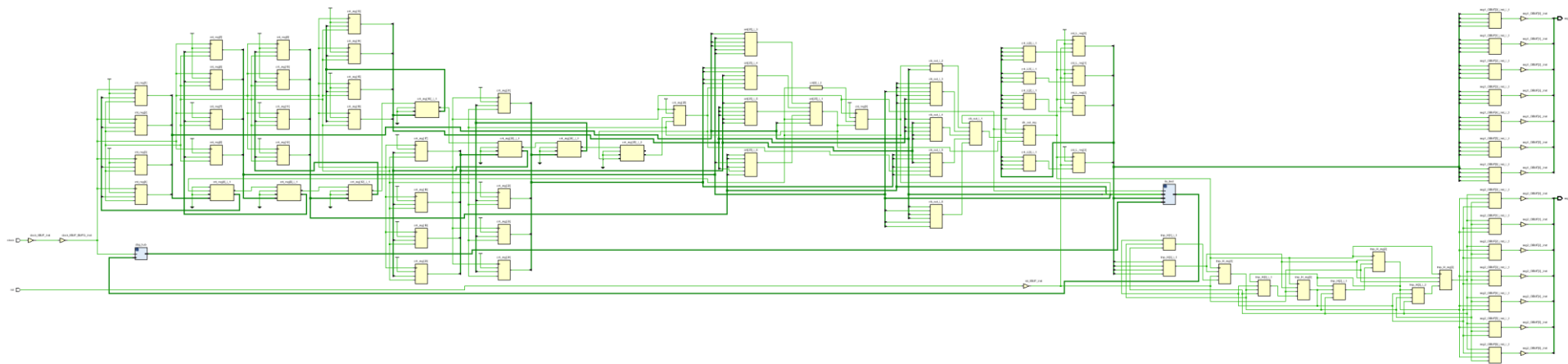
[illegible]

□ 插入ILA

```
23 module clock (  
24     input wire clock,  
25     input wire rst,  
26     output reg[6:0] seg1,  
27     output reg[6:0] seg2  
28 );  
29  
30     reg[3:0] cnt_L = 4'b0000;  
31     reg[3:0] cnt_H = 4'b0000;  
32     reg[25:0] cnt = 26'b000000000000000000000000;  
33     reg clk_out;  
34     reg[3:0] tmp_H = 4'b0000;  
35     reg[3:0] tmp_L = 4'b0000;  
36  
37     ila_0 ila_test(  
38         .clk(clock),  
39         .probe0(cnt[9:0]),  
40         .probe1(cnt_L)  
41     );  
42  
43     always_ff @ (posedge clock) begin  
44         cnt <= cnt + 1;
```

重新综合实现





□ 连接ILA

调试工具

内存总线分析

远程 JTAG 连接

内部信号采样

连接方法

本功能允许 Vivado 远程连接到 FPGA 的 JTAG，以便支持 ILA 和 VIO 等功能。

Hardware Server IP 166.111.226.111, Port 4192

[连接方法说明](#)

调试工具

内存总线分析

远程 JTAG 连接

内部信号采样

连接方法

本功能允许 Vivado 远程连接到 FPGA 的 JTAG，以便支持 ILA 和 VIO 等功能。

Hardware Server IP 166.111.226.111, Port 4192

PROGRAM AND DEBUG

- Generate Bitstream
- Open Hardware Manager
 - Open Target

Hardware Server Settings

Connect to: Remote server (target is on remote machine)

Remote Server 填写网页上显示的IP和端口

Host name: 166.111.227.237

Port: 1234 [default is 3121]

Click Next to launch and/or connect to the hw_server (port 1234) application on the remote machine '166.111.227.237'.

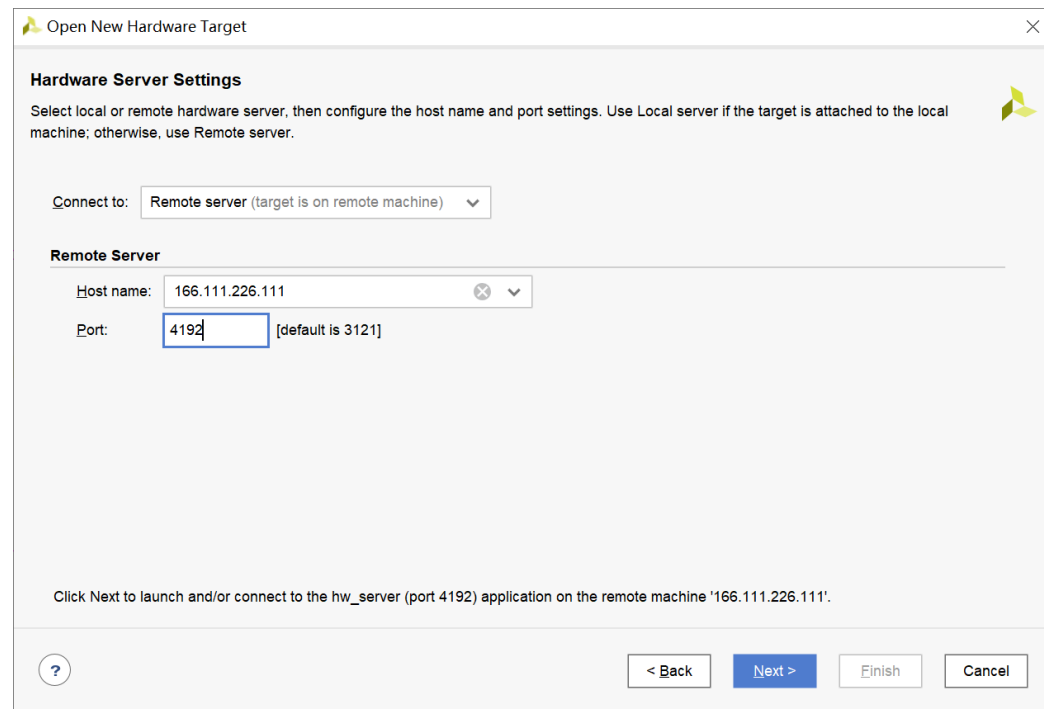
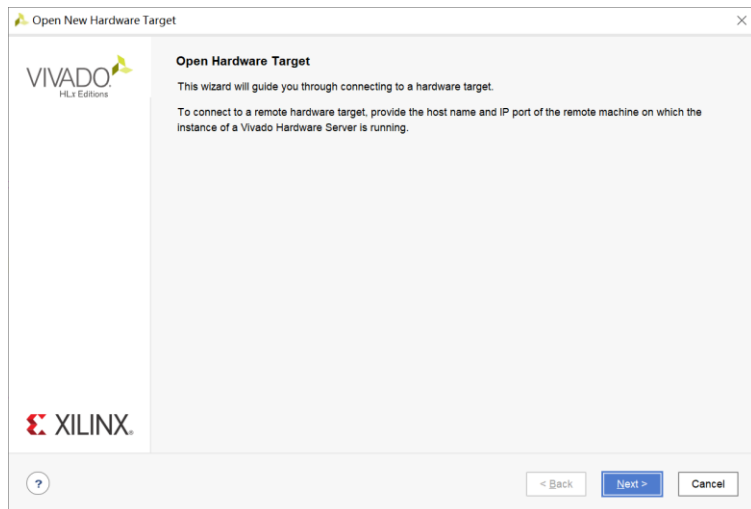
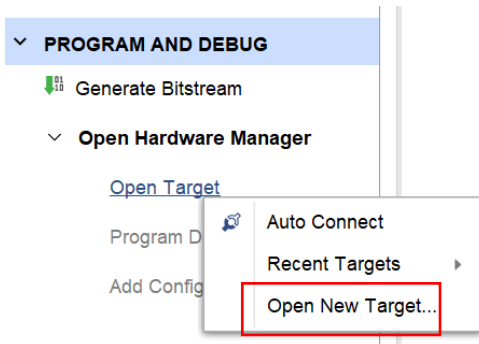
Select Hardware Target

Select a hardware target from the list of available targets, then set the appropriate JTAG clock (TCK) frequency. If you do not see

Type	Name	JTAG Clock Frequency
xilinx_tcf	Xilinx/1.0.2.30.35.2542	100000000

如果连接成功，这一步将显示连接上目标点下一步即可

Hardware server: 166.111.227.237:4498



Open New Hardware Target

Select Hardware Target

Select a hardware target from the list of available targets, then set the appropriate JTAG clock (TCK) frequency. If you do not see the expected devices, decrease the frequency or select a different target.

Hardware Targets

Type	Name	JTAG Clock Frequency
xilinx_tcf	Xilinx/10.2.0.30:2542	10000000

Add Xilinx Virtual Cable (XVC)

Hardware Devices (for unknown devices, specify the Instruction Register (IR) length)

Name	ID Code	IR Length
xc7a100t_0	13631093	6

Hardware server: 166.111.226.111:4192

硬件服务器

?

< Back

Next >

Finish

Cancel

Flow Navigator

HARDWARE MANAGER - 166.111.226.111/xilinx_tcf/Xilinx/10.2.0.30.2542

PROJECT MANAGER

- Settings
- Add Sources
- Language Templates
- IP Catalog

IP INTEGRATOR

- Create Block Design
- Open Block Design
- Generate Block Design

SIMULATION

- Run Simulation

RTL ANALYSIS

- Open Elaborated Design

SYNTHESIS

- Run Synthesis
- Open Synthesized Design

IMPLEMENTATION

- Run Implementation
- Open Implemented Design

PROGRAM AND DEBUG

- Generate Bitstream
- Open Hardware Manager
 - Open Target
 - Program Device
 - Add Configuration Memory Dev

Hardware

Name	Status
166.111.226.111 (1)	Connected
xilinx_tcf/Xilinx/10.2	Open
xc7a100t_0 (2)	Programmed
XADC (System)	
hw_ila_1 (ila_1)	Idle

Hardware Device Properties

xc7a100t_0

Name: xc7a100t_0

Part: xc7a100t

ID code: 13631093

IR length: 6

Status: Programmed

Programming file: :ts/clock/clock.runs/impl_1/clock.bit

Probes file: :ts/clock/clock.runs/impl_1/clock.ltx

User chain count: 4

General Properties

Waveform - hw_ila_1

ILA Status: Idle

Name	Value
cnt[3:0]	
cnt_L[3:0]	

Settings - hw_ila_1

Status - hw_ila_1

Core status: Idle

Capture status - Window 1 of 1

Window sample 0 of 1024

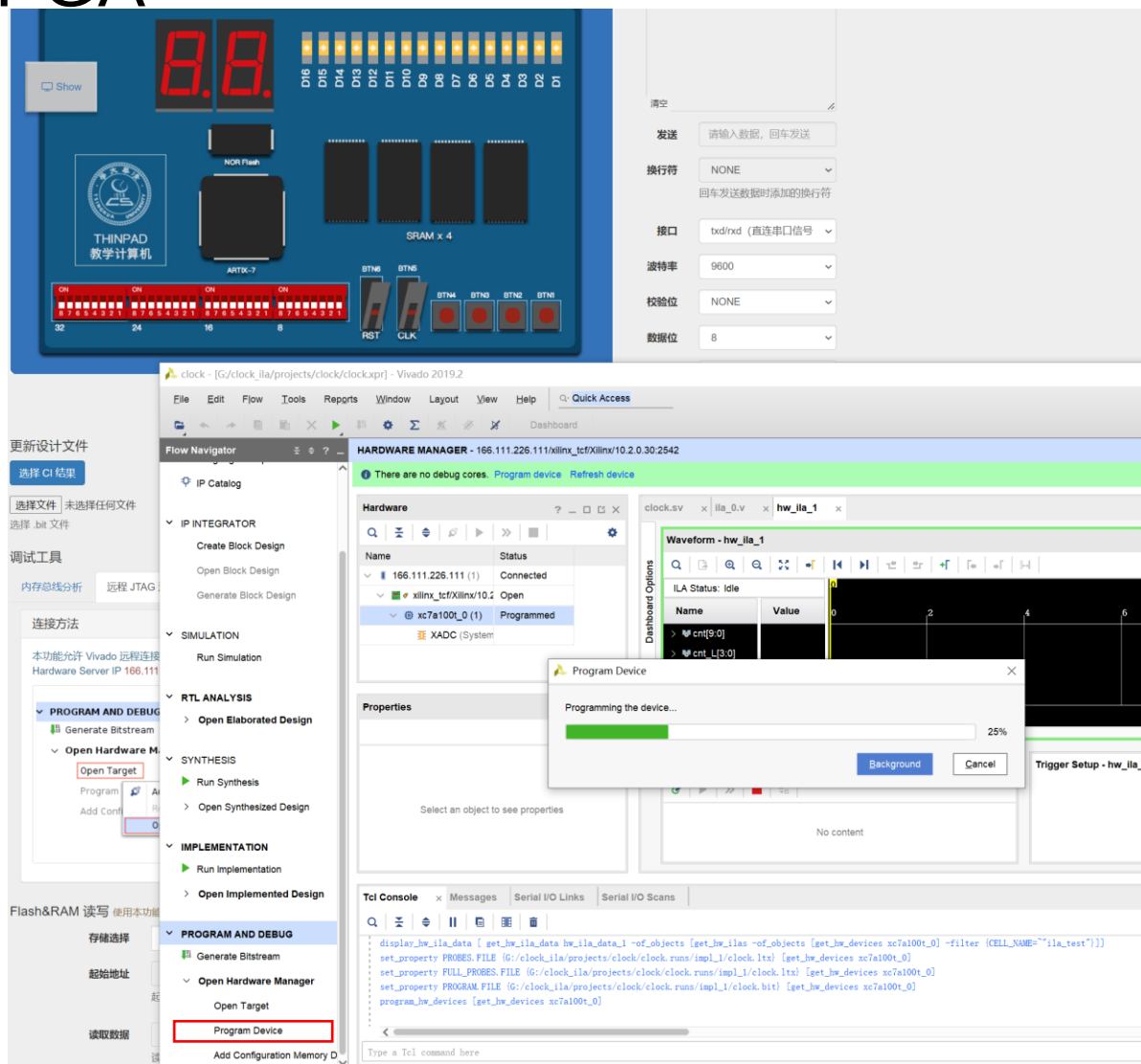
Idle

Trigger Setup - hw_ila_1

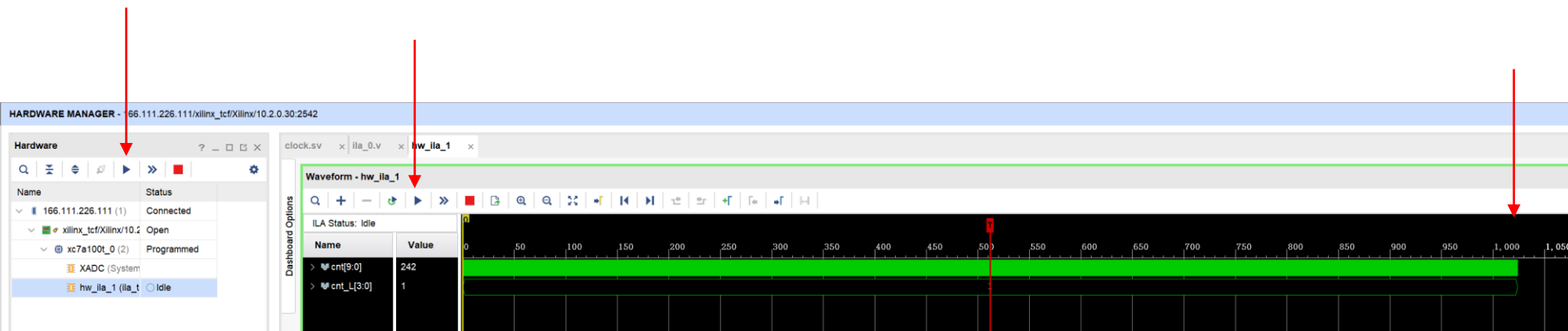
Capture Setup - hw_ila_1

Press the + button to add probes.

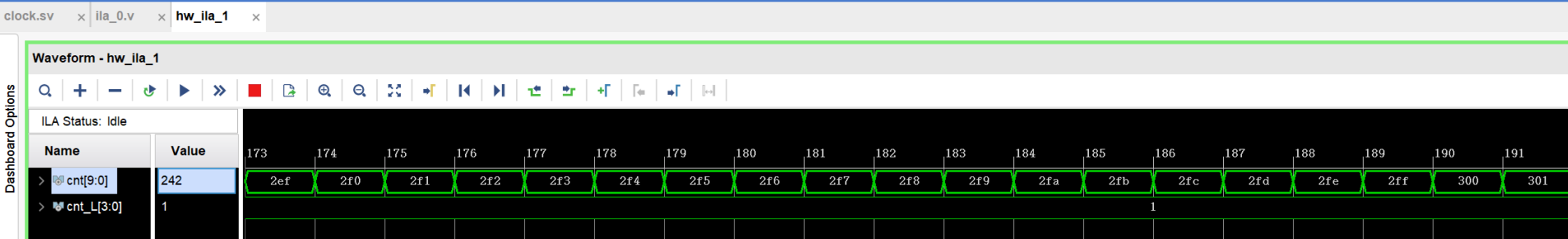
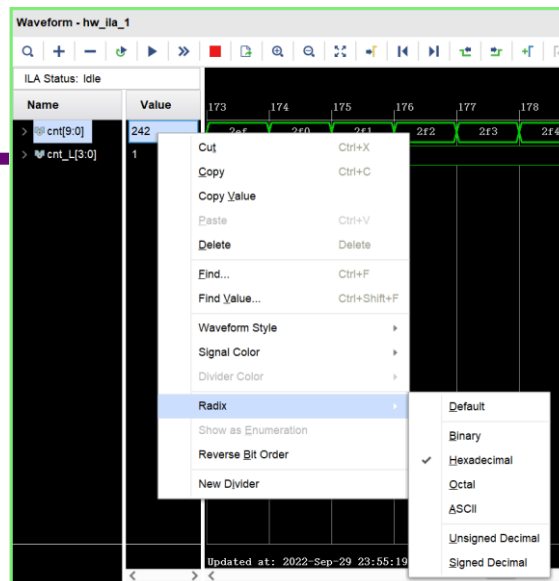
重新下载FPGA



□ 采样数据



查看数据

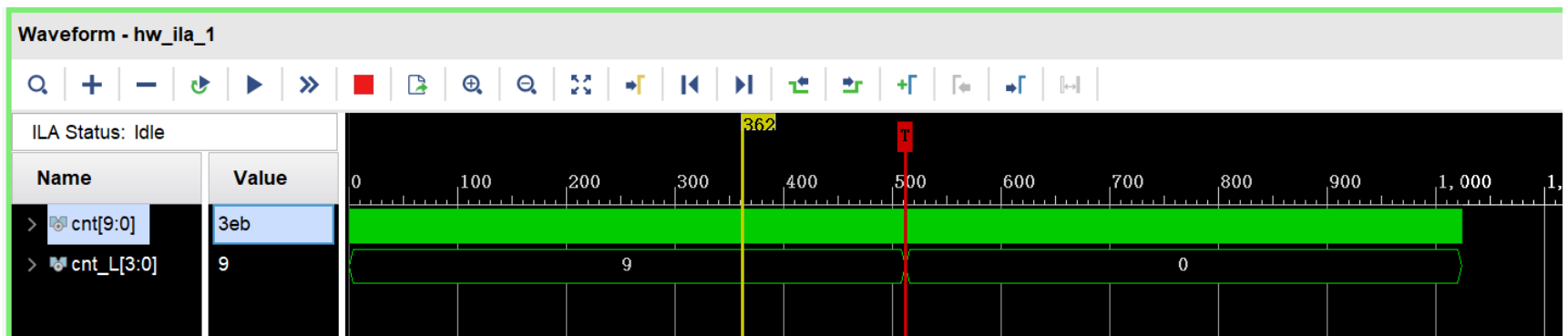


□ 设置trigger

Trigger Setup - hw_ila_1 x Capture Setup - hw_ila_1

Q + - D

Name	Operator	Radix	Value	Port	Comparator Usage
cnt_L[3:0]	==	[H]	0	probe1[3:0]	1 of 1



谢谢